

VIO + ILA over JTAG

Driving and observing the 4×4 weight-stationary systolic-array multiplier on a Digilent ZedBoard entirely from the Vivado Hardware Manager

Target part: `xc7z020c1g484-1` · Vivado 2019.1 or later · Design under test: `accelerator_system_top` (unmodified) · Companion document: `USB_UART.pdf`

Contents

- | | |
|--|--|
| 1. What this method gives you | 10. Programming the board and opening the dashboards |
| 2. Prerequisites and board setup | 11. Running the demo from the VIO dashboard (manual) |
| 3. The design under test — everything the harness must respect | 12. Running the demo from Tcl (scripted) |
| 4. The debug harness <code>vio_debug_top.sv</code> | 13. Reading the ILA waveform — the expected cycle map |
| 5. Vivado project creation | 14. Trigger recipes |
| 6. Generating the VIO core | 15. Resource and timing — measured |
| 7. Generating the ILA core | 16. Troubleshooting |
| 8. Probing internal nets without editing the RTL | 17. Extending the harness |
| 9. Constraints, synthesis, implementation, bitstream | 18. Writing an XDC for any design — a general guide |
| | 19. Appendix A — data packing reference |
| | 20. Appendix B — full source listing |

Plus §8A (between §8 and §9) — **How the VIO and ILA get wired in, and how the build is staged:** the two ways a debug core enters a design, what IP generation really produces, the auto-inserted `dbg_hub` and its BSCANE2/JTAG path, the auto-generated `connect_debug_port` constraints, the ten build stages from IP synthesis to programming, where every artefact lands on disk, and how to verify the wiring at each stage.

REVISION NOTE — READ THIS IF YOU USED THE FIRST EDITION

Three corrections have been folded into this document, all driven by running the flow rather than reasoning about it:

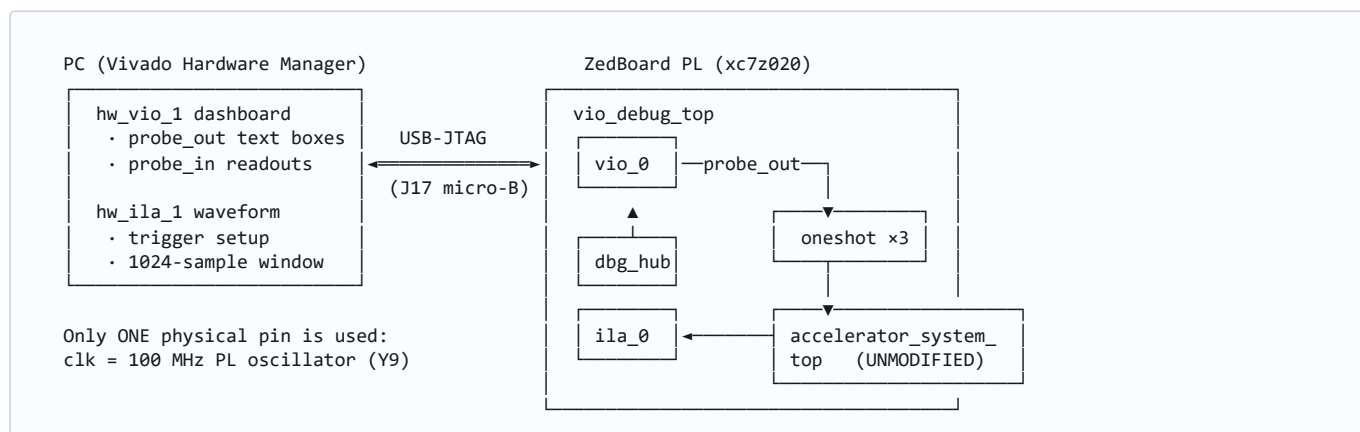
- **§12 — THE TCL SCRIPT HAD A REAL BUG.**
`accel_read` returned the four result lanes *reversed* (4 3 2 1 where the golden is 1 2 3 4), so `accel_demo` reported MISMATCH on every row even on correct hardware. Fixed, with the error modes now documented in §12.4.
- **§11.7 — RESULT_FLAT READS ALL-ZERO ON THE VIO DASHBOARD, AND THAT IS CORRECT BEHAVIOUR, NOT A FAULT.**
 It is a live bus that only carries data for four clocks out of a 170 ns job. Full explanation added.
- **§15 — THE TIMING CLAIM WAS OVERSTATED.**
 Measured post-synthesis WNS is **+1.279 NS**
 at 100 MHz, not the "enormous" margin previously claimed. Real utilization figures now replace the estimates.

1. What this method gives you

The accelerator finishes a four-vector job in **17 clock cycles = 170 ns** at 100 MHz. That is far too fast for a human to watch on LEDs, and eight switches cannot hold a 4×4 int8 matrix. Method 1 (switches + LEDs) works around this with a byte-at-a-time console FSM. **Method 2 removes the board from the loop entirely:** the JTAG programming cable carries both the stimulus and the observation.

Two Xilinx debug cores do the work:

Core	Role	What it buys you here
VIO Virtual I/O	Software-driven inputs and outputs, updated at human speed over JTAG	Type a 32-bit weight word into a text box and click <i>commit</i> — no switches, no button debouncing, no 16-press entry sequence. Read back the full 128-bit result word as one value.
ILA Integrated Logic Analyzer	On-chip trace buffer, samples every signal on every clock edge	The only way to see the pipeline <i>as it runs on silicon</i> : the four WLOAD cycles, the six the diagonal wavefront through COMPUTE, and each column's result landing in <code>output_mem</code> during DRAIN.



THE RULE THAT SHAPES EVERYTHING

The 13 RTL files in `...SIMPLE_ACCEL_RTL/` are

NEVER MODIFIED

. Every technique in this document either wraps the design (`vio_debug_top.sv`) or applies a *constraint* to the synthesized netlist (`MARK_DEBUG` from the XDC). No `(* mark_debug *)` attribute is ever added to a design source.

1.1 Deliverables this document uses

File	Purpose
<code>FPGA_IMPL/rtl/vio_debug_top.sv</code>	The debug top: VIO + ILA + one-shot pulse generators + sticky-done around <code>accelerator_system_top</code>
<code>FPGA_IMPL/vio_debug.xdc</code>	Clock pin, clock period, debug-hub settings, optional <code>MARK_DEBUG</code> lines
<code>FPGA_IMPL/vio_demo.tcl</code>	Hardware Manager script: loads the T5 golden matrices, runs the job, reads back and checks all 16 results
<code>...SIMPLE_ACCEL_RTL/*.sv</code>	The 13 unmodified design files

2. Prerequisites and board setup

2.1 Tools

- **Vivado 2019.1 or later**, any edition that supports Zynq-7000 (the free WebPACK covers `xc7z020`). Vitis is *not* needed for this method — no software runs on the ARM.
- **Digilent Adept / cable drivers**. On Windows, Vivado's installer offers "Install Cable Drivers"; accept it. Verify afterwards that Device Manager shows the JTAG interface without a warning triangle.
- Optionally the **Digilent board files** for the ZedBoard, which let you pick the board by name instead of the raw part. Not required — selecting the part directly works fine.

2.2 Board jumpers and cables

Item	Setting	Why
Power source JP3	USB (or wall adapter)	Either is fine; USB power is adequate for a PL-only design.
Boot mode JP7–JP11	All five to GND = JTAG mode	Stops the PS from trying to boot from QSPI/SD. In JTAG mode the PS sits idle and the PL TAP is reachable. Verify against the ZedBoard Hardware User Guide for your board revision.
USB cable	Micro-USB into J17 (PROG)	J17 is the JTAG/programming port. J14 is the USB-UART and is <i>not</i> used by this method.
Power switch SW8	ON	The green PG00D LED (LD13) must be lit.

PL-ONLY ON A ZYNQ IS LEGAL

This design never configures the Processing System. That is supported: the PL is programmed straight over JTAG and clocked by the 100 MHz board oscillator on pin Y9, entirely independent of the PS. Vivado will emit an *unconstrained PS pins* DRC; the supplied XDC silences it with `set_property BITSTREAM.GENERAL.UNCONSTRAINEDPINS {Allow}`. In cascaded JTAG mode the Hardware Manager shows

TWO

devices in the chain — `arm_dap_0` and `xc7z020_1`. Program the `xc7z020_1`.

3. The design under test — everything the harness must respect

Read this section before building anything. Four properties of `accelerator_system_top` completely determine the shape of the debug harness.

3.1 Port list

Port	Dir	Width	Meaning
clk	in	1	System clock (100 MHz here)
rst_n	in	1	Active-low synchronous reset
start	in	1	Launch a job. Sampled only in IDLE ; ignored while busy.
num_vectors	in	8	How many activation vectors to stream. 0 is legal and does not hang.
act_base	in	8	First <code>a_mem</code> address of the job
out_base	in	8	First <code>output_mem</code> address of the job
host_a_we / _addr / _wdata	in	1 / 8 / 32	Activation-memory write port. Honoured only while busy==0 .
host_w_we / _addr / _wdata	in	1 / 2 / 32	Weight-memory write port. Honoured only while busy==0 .
host_o_addr	in	8	Result read address (host side of <code>output_mem</code> is read-only)
o_rdata	out	128	Result word: four int32 lanes, lane <code>c</code> = column <code>c</code>
phase	out	3	FSM state: 0 IDLE, 1 WLOAD, 2 COMPUTE, 3 DRAIN, 4 DONE
busy	out	1	<code>(state != IDLE) && (state != DONE)</code>
done	out	1	Single-cycle pulse. <code>busy</code> is already low when it fires.
result[0:3]	out	4×32	Live result bus, for observation. Unpacked array — must be flattened to probe it.

3.2 Host protocol

```
1. hold rst_n low, then release
2. while (!busy): write weight rows via host_w_* (4 words)
                   write activations via host_a_* (num_vectors words)
3. pulse start, with num_vectors / act_base / out_base valid
4. wait for the done pulse (job length = num_vectors + 13 clocks)
5. while (!busy): read results via host_o_addr -> o_rdata
```

3.3 The four properties that shape the harness

#	Property	Consequence for a VIO-driven design
1	Every command is a single-cycle pulse. <code>start</code> , <code>host_w_we</code> , <code>host_a_we</code> .	A VIO <code>probe_out</code> is a <i>level</i> — it stays where you left it for milliseconds until you click again. Holding <code>start</code> high would relaunch the 17-cycle job roughly 4 million times per second; the ILA would retrigger continuously and never let you inspect a single run. The harness edge-detects every command bit (<code>oneshot</code> module).
2	done is one cycle wide (10 ns).	VIO refreshes at human rate and will never observe it. The harness latches it into <code>done_sticky</code> , cleared automatically by the next <code>start</code> pulse. A 16-bit <code>done_count</code> is also provided so you can prove exactly one job ran per click.
3	Host writes are masked while busy==1 . Port arbitration at L3 forces <code>a_we_mux = w_we_mux = 0</code> during a job.	Harmless but silent: a write clicked mid-job simply vanishes. Always confirm <code>busy==0</code> on the dashboard before loading. Because JTAG round-trips take milliseconds and a job takes 170 ns, this is essentially never a problem in practice.
4	Fixed latency, no back-pressure. No ready/valid handshake exists anywhere in the design.	Nothing will stall to wait for you. Every timing number in section 13 is a hard constant, which is exactly what makes the ILA capture checkable against the specification.

3.4 Data layout — get a lane wrong and the matrix silently transposes

```
weight_mem[r] [31:24][23:16][15:8][7:0] = W[r][3] W[r][2] W[r][1] W[r][0] (lane c = column c)
a_mem[i]       [31:24][23:16][15:8][7:0] = A[i][3] A[i][2] A[i][1] A[i][0] (lane r = row r)
output_mem[i]  [127:96][95:64][63:32][31:0] = C[i][3] C[i][2] C[i][1] C[i][0] (lane c, int32)
```

THE BOTTOM-FIRST RULE IS NOT YOURS TO APPLY

The design's rule 1 says "weight rows load bottom-first". That is the

AGU'S

job — during WLOAD it counts `w_addr` down 3, 2, 1, 0. As the host you write `weight_mem[r] = W` row `r` in

NATURAL ORDER 0, 1, 2, 3

. Pre-reversing them here is a classic and silent error: the result is a plausible-looking but wrong matrix.

4. The debug harness `vio_debug_top.sv`

The harness is deliberately thin. It adds exactly four things around the unmodified accelerator:

1. **A power-on reset.** A 16-bit counter holds `rst_n` low for 65 536 clocks ($\approx 655\ \mu\text{s}$) after configuration. The declaration initializer (`logic [15:0] por = '0;`) is a synthesizable power-up value on Xilinx 7-series — flip-flops configure to a known state at bitstream load, so no external reset pin is required.
2. **Three oneshot instances.** Rising-edge detectors on `vio_start_lv1`, `vio_w_we_lv1`, `vio_a_we_lv1`. Property 1 above.
3. **Sticky done + a job counter.** Property 2 above.
4. **A flattening assign.** `result[0:3]` is an unpacked array and cannot be wired to a probe port; `result_flat` concatenates it into one 128-bit vector.

```
// the load-bearing 10 lines of the harness

// level -> single-cycle pulse (property 1)
oneshot u_os_start (.clk(clk), .rst_n(rst_n), .lv1(vio_start_lv1), .pulse(start_p));
oneshot u_os_wwe (.clk(clk), .rst_n(rst_n), .lv1(vio_w_we_lv1), .pulse(w_we_p));
oneshot u_os_awe (.clk(clk), .rst_n(rst_n), .lv1(vio_a_we_lv1), .pulse(a_we_p));

// catch the 10 ns done pulse (property 2)
if (start_p) done_sticky <= 1'b0; // arm for the new job
else if (done) done_sticky <= 1'b1; // latch
if (done) done_count <= done_count + 1'b1;

// unpacked array -> probeable vector
assign result_flat = {result[3], result[2], result[1], result[0]};
```

WHY ONESHOT IS NOT OPTIONAL

Without it the design still *computes correctly* — the job is idempotent, so re-running it gives the same answer. What breaks is

OBSERVABILITY

: `busy` never reads 0 on the dashboard, `done_sticky` is cleared as fast as it is set, and the ILA arms and fires continuously so you can never freeze a single 17-cycle window. It is the difference between a working demo and an unreadable one.

The full listing is in Appendix B. The two IP instantiations at the bottom (`vio_0` and `ila_0`) refer to cores you generate in sections 6 and 7 — their port widths must match the instance exactly or elaboration fails.

5. Vivado project creation

Step 5.1 — Create the project.

Vivado → *Create Project* → *Next* → name it e.g. `accel_vio`, choose a path with **no spaces** → *Next* → **RTL Project**, tick *Do not specify sources at this time* → *Next* → **Parts** tab → filter and select `xc7z020c1g484-1` (or the *Boards* tab → ZedBoard, if you installed the board files) → *Next* → *Finish*.

Step 5.2 — Add the design sources.

Flow Navigator → *Add Sources* → *Add or create design sources* → *Add Files*. Select:

- all **13** `.sv` files in `...SIMPLE_ACCEL_RTL/`: `MAC.sv`, `array.sv`, `delay_pipe.sv`, `input_buf.sv`, `output_skewbuf.sv`, `a_mem.sv`, `weight_mem.sv`, `outmem.sv`, `controller.sv`, `agu.sv`, `wrapper_array_buf.sv`, `wrapper_mem_arr_buf.sv`, `wrapper_full_system.sv`
- `FPGA_IMPL/rtl/vio_debug_top.sv`

Tick **Copy sources into project** if you want the project self-contained. Click *Finish*.

CHECK THE FILE TYPE

In the *Sources* pane, confirm every file shows as

SYSTEMVERILOG

, not Verilog. If any is mis-typed, right-click → *Set File Type* → *SystemVerilog*. Symptom of getting this wrong: a flood of syntax errors on `logic`, `always_ff`, or the unpacked-array port `result [0:3]`.

Step 5.3 — Set the top module.

Right-click `vio_debug_top` in the *Sources* hierarchy → **Set as Top**. It turns bold. At this point the hierarchy will show `vio_0` and `ila_0` as *missing* — that is expected; you generate them next.

Step 5.4 — Add the constraints.

Add Sources → *Add or create constraints* → *Add Files* → `FPGA_IMPL/vio_debug.xdc`.

6. Generating the VIO core

Flow Navigator → *IP Catalog* → search `VIO` → double-click **VIO (Virtual Input/Output)**.

6.1 General Options tab

Field	Value	Note
Component Name	<code>vio_0</code>	Must match the instance name used in <code>vio_debug_top.sv</code> .
Input Probe Count	6	Ports <code>probe_in0</code> ... <code>probe_in5</code>
Output Probe Count	11	Ports <code>probe_out0</code> ... <code>probe_out10</code>

6.2 PROBE_IN Ports tab — widths

Port	Width	Signal	What you read on the dashboard
<code>probe_in0</code>	1	<code>busy</code>	0 = the accelerator is idle and will honour host writes
<code>probe_in1</code>	1	<code>done_sticky</code>	1 = a job has completed since the last <code>start</code>
<code>probe_in2</code>	3	<code>phase</code>	Frozen at 0 (IDLE) whenever you look — the job is 170 ns long
<code>probe_in3</code>	128	<code>o_rdata</code>	This is where you read results. The stored result word selected by <code>host_o_addr</code> — persistent, readable any time after the job.
<code>probe_in4</code>	128	<code>result_flat</code>	Will always read 0 here — see §11.7. It is a live bus valid for only 4 clocks out of a 170 ns job, so VIO can never catch it. Meaningful only on the ILA (<code>probe8</code>).
<code>probe_in5</code>	16	<code>done_count</code>	Total jobs completed since reset — the cheapest "did it run?" check

6.3 PROBE_OUT Ports tab — widths and initial values

Port	Width	Init	Signal	Meaning
probe_out0	1	0	vio_start_lvl	0→1 launches one job
probe_out1	8	0x04	vio_num_vectors	Pre-set to 4 so the T5 demo works out of the box
probe_out2	8	0x00	vio_act_base	First <code>a_mem</code> address
probe_out3	8	0x00	vio_out_base	First <code>output_mem</code> address
probe_out4	32	0	vio_w_wdata	Weight word (four int8 lanes)
probe_out5	2	0	vio_w_addr	Weight row 0–3
probe_out6	1	0	vio_w_we_lvl	0→1 commits one weight word
probe_out7	32	0	vio_a_wdata	Activation word (four int8 lanes)
probe_out8	8	0	vio_a_addr	Activation index 0–255
probe_out9	1	0	vio_a_we_lvl	0→1 commits one activation word
probe_out10	8	0	vio_o_addr	Which result word <code>o_rdata</code> shows

Click **OK** → **Generate** → *Out of context per IP* (the default) → *Generate*.

WHY IS THE ACTIVATION ADDRESS 8 BITS BUT THE WEIGHT ADDRESS ONLY 2?

This looks like an inconsistency the first time you see the dashboard — both matrices are 4×4 , and for this demo you write exactly four words to each. It is not a mistake, and it is not something you should "fix". The two address buses are sized for the

MEMORIES BEHIND THEM

, and those memories differ in depth by $64\times$:

Probe	Width	Memory	Geometry	Why that depth
<code>vio_w_addr</code>	2	<code>weight_mem</code>	4×32	Holds ONE WHOLE MATRIX, ONCE . The array has ROWS = 4 rows; each 32-bit word packs the four int8 weights of one row. Four rows $\Rightarrow$ four words $\Rightarrow$ 2 address bits. Weights are loaded into the 16 PEs once per job and stay resident. A deeper weight memory would be dead silicon.
<code>vio_a_addr</code>	8	<code>a_mem</code>	256×32	Holds a WHOLE STREAM of activation vectors. 256 words = up to 256 vectors multiplied against a SINGLE weight load without reloading weights.

That asymmetry *is* the weight-stationary argument, made concrete in the port widths: weight traffic is ROWS $\times$ COLS bytes

PER JOB

, activation traffic is ROWS bytes

PER VECTOR

. You pay the weight-load cost once and amortise it over many activation rows. An 8-bit weight address would be $64\times$ wider than anything the design can use; a 2-bit activation address would cap you at four vectors and throw away the entire economic point of the architecture.

`vio_o_addr` is likewise

8

bits — `output_mem` is 256×128 , mirroring `a_mem`'s depth, because the design produces one result vector per activation vector.

INITIAL VALUES MATTER

A VIO `probe_out` comes up at its configured initial value after configuration, and Vivado's dashboard *also* re-applies stored values when you reopen it. Setting `probe_out1 = 0x04` means a freshly programmed board is already configured for a four-vector job — one less thing to get wrong. Leave every command bit (`probe_out0/6/9`) at

0

so nothing fires at power-up.

7. Generating the ILA core

IP Catalog $\rightarrow$ search `ILA` $\rightarrow$ double-click **ILA (Integrated Logic Analyzer)**.

7.1 General Options tab

Field	Value	Note
Component Name	ila_0	Must match the instance name.
Monitor Type	Native	Not AXI — we probe raw signals, not a bus interface.
Number of Probes	10	probe0 ... probe9
Sample Data Depth	1024	10.24 μs of trace at 100 MHz. The job is only 17 cycles, so 1024 gives you generous pre- and post-trigger context. See the BRAM note below.
Same Number of Comparators for All Probes	ticked	Simplest configuration; 1 comparator per probe is enough for the triggers in section 14.
Trigger Out / Trigger In ports	unticked	Not needed — a single ILA, self-triggered.
Input Pipe Stages	0	Timing has enormous margin here; extra pipe stages would only shift samples relative to each other and confuse the cycle map in section 13.
Capture Control	optional	Tick it only if you want the storage qualifier described in section 14.3.

7.2 Probe_Ports tab — widths

Port	Width	Signal	Why it is worth a probe
probe0	1	start_p	The trigger. One clock wide, exactly once per dashboard click.
probe1	1	busy	Marks the job window. Should be high for exactly 22 samples on a 4-vector job.
probe2	1	done	The real 10 ns pulse — the ILA is the only thing fast enough to see it.
probe3	3	phase	The centrepiece. Phase-length counting is the primary hardware/spec cross-check.
probe4	1	w_we_p	Confirms the one-shot produced exactly one clock per weight write.
probe5	1	a_we_p	Same for activation writes.
probe6	8	vio_a_addr	Which activation slot the last write targeted.
probe7	128	o_rdata	Result read-back path.
probe8	128	result_flat	The live result bus. Watch C0...C3 emerge one per clock during DRAIN.
probe9	16	done_count	Increments by exactly 1 inside the window — proves one job, not a burst.

Click **OK** → **Generate**.

SIZING THE TRACE BUFFER

Total probe width = 1+1+1+3+1+1+8+128+128+16 =

288 BITS

. At depth 1024 that is 288 × 1024 = 294 912 bits ≈

8 BRAM36

on xc7z020 (140 available), plus a little for the trigger logic. If you are tight on block RAM, or you only care about the job itself, drop

SAMPLE DATA DEPTH TO 256

— the whole 17-cycle job still fits with room to spare, and the cost falls to ~2 BRAM36. Alternatively drop probe7 (o_rdata), since you can read the same data from the VIO dashboard after the run.

8. Probing internal nets without editing the RTL

The ILA instantiated in `vio_debug_top` can only see signals visible at the harness level. To watch what happens *inside* the accelerator — the AGU's `wload` and `valid_in` strobes, the address counters, the partial sums crossing the PE array — you need netlist nets. Since the RTL must not be edited, there are two non-invasive routes.

8.1 Route A — the Set Up Debug wizard RECOMMENDED FOR EXPLORING

1. Run **Synthesis** once and *Open Synthesized Design*.
2. In the **Netlist** pane, expand `vio_debug_top` → `u_acc`. Right-click nets of interest — `agu_wload`, `agu_valid_in`, `agu_out_we`, `agu_w_addr[1:0]`, `agu_act_addr[7:0]`, `agu_out_addr[7:0]` — and choose **Mark Debug**.
3. *Tools* → *Set Up Debug* → *Next* → review the net list → *Next* → confirm the **debug clock domain is** `clk` for every net → set **sample depth 1024** → *Next* → *Finish*.
4. Vivado writes `create_debug_core` / `connect_debug_port` commands into your XDC. Save it. This inserts a *second* ILA (`u_ila_0`) alongside your instantiated `ila_0`; both appear in the Hardware Manager as separate dashboards.
5. Re-run **Synthesis** → **Implementation** → **Generate Bitstream**.

8.2 Route B — `MARK_DEBUG` from the XDC RECOMMENDED FOR REPEATABILITY

The wizard's output is verbose and tied to one particular netlist. For a setup you want to reproduce, put the marking in your own constraints file instead. `vio_debug.xdc` ships with these lines commented out:

```
set_property MARK_DEBUG true [get_nets u_acc/agu_wload]
set_property MARK_DEBUG true [get_nets u_acc/agu_valid_in]
set_property MARK_DEBUG true [get_nets u_acc/agu_out_we]
set_property MARK_DEBUG true [get_nets {u_acc/agu_w_addr[*]}]
set_property MARK_DEBUG true [get_nets {u_acc/agu_act_addr[*]}]
set_property MARK_DEBUG true [get_nets {u_acc/agu_out_addr[*]}]
```

FIND THE REAL NET NAMES FIRST — DO NOT GUESS

Synthesis renames, merges and absorbs nets. Before uncommenting anything, open the Synthesized Design and run these in the Tcl console:

```
get_nets -hierarchical *agu_wload*
get_nets -hierarchical *agu_valid_in*
get_nets -hierarchical *agu_out_we*
```

Paste back whatever they actually return. An empty result means the net was optimized into its consumer — see the next box.

PRESERVE HIERARCHY SO THE NAMES SURVIVE

Settings → *Synthesis* → *-flatten_hierarchy* → set to

NONE

. The default (`rebuilt`) flattens for optimization and then reconstructs names, which is usually good enough but occasionally loses a net. Setting `none` guarantees the hierarchical names above resolve. It does cost some QoR — with a measured baseline of WNS +1.279 ns (\$15) there is room for that, but check timing after changing it rather than assuming. If a specific net still disappears, add `set_property KEEP true [get_nets ...]` or `set_property DONT_TOUCH true [get_cells u_acc/u_agu]`.

8.3 What is worth probing inside the design

Net (inside <code>u_acc</code>)	What it proves
<code>agu_wload</code>	High for exactly 4 cycles during WLOAD, then low. Together with <code>agu_w_addr</code> it demonstrates the bottom-first weight ordering on real silicon.
<code>agu_w_addr[1:0]</code>	Counts 3, 2, 1, 0 during WLOAD — the single most misunderstood behaviour in the design.
<code>agu_valid_in</code>	High for exactly <code>num_vectors</code> cycles during COMPUTE, one activation vector per clock.
<code>agu_act_addr[7:0]</code>	Walks <code>act_base</code> , <code>act_base+1</code> , ... Confirms <code>act_base</code> is honoured.
<code>agu_out_we</code> / <code>agu_out_addr</code>	Four write strobes landing at STORE_LATENCY = 8 cycles after each activation address was issued — i.e. inside DRAIN. This is the timing derivation made visible.
<code>u_acc/u_datapath/.../psum_out_bottom</code>	Deep inside the array: the partial sums leaving the bottom PE row. Expensive in probes; only worth it if you are debugging the array itself rather than the system.

8A. How the VIO and ILA get wired in, and how the build is staged

Sections 6 and 7 told you which boxes to tick in the IP wizards, and section 4 showed `vio_debug_top.sv` instantiating `vio_0` and `ila_0` by name. What neither explained is the part that confuses almost everyone the first time: **you never instantiate the debug hub, you never write a single line of JTAG logic, and you never connect the cores to the outside world — yet the Hardware Manager finds them.** This section explains exactly what Vivado does on your behalf, at which stage of the build it does it, and how to verify each step.

8A.1 There are two completely different ways a debug core enters a design

Both end up in the same place. Confusing them is the source of most "why is my ILA not there" questions.

	Path A — RTL instantiation USED HERE	Path B — netlist insertion (<code>MARK_DEBUG</code> / <i>Set Up Debug</i>)
How the core is created	You generate <code>vio_0</code> / <code>ila_0</code> from the IP catalog, then write <code>vio_0 u_vio (...)</code> in your HDL.	You mark nets for debug and run <i>Tools</i> → <i>Set Up Debug</i> . Vivado creates the core itself, in the netlist, after synthesis.
Where the connections are described	In your source. <code>.probe_in0(busy)</code> , <code>.probe_out0(vio_start_lvl)</code> — the port map you wrote in <code>vio_debug_top.sv</code> .	In an auto-generated XDC. <code>create_debug_core</code> , <code>create_debug_port</code> , <code>connect_debug_port</code> commands the wizard writes for you.
Visible in the RTL hierarchy?	Yes — <code>u_vio</code> , <code>u_ila</code> appear in the Sources tree.	No. The core exists only from <code>opt_design</code> onward.
Requires re-synthesis to change	Yes — you edited HDL.	Yes if the marks are on RTL nets (the <code>MARK_DEBUG</code> property is applied during synthesis), no if you only re-run the wizard on an existing synthesized netlist.
Best for	A permanent, scripted, reproducible harness — a bring-up rig you will use many times. Also the only way to get a VIO's <code>probe_out</code> to drive logic , because outputs must be connected to something in your design.	Opportunistic debugging: "I need to see this net, once." Also the only non-invasive way to probe inside a module you are not allowed to edit — which is why section 8 uses it for the AGU nets.

THIS PROJECT DELIBERATELY USES BOTH

Path A for the harness — the VIO and ILA are instantiated in `vio_debug_top.sv`, so the whole rig is reproducible from source and can be version-controlled. Path B for the internal AGU nets, because the 13 RTL files must never be edited and `(* mark_debug *)` cannot be added to them. The two mechanisms coexist in one build without any special handling.

8A.2 Path A in detail — what "generating the IP" actually produces

When you click *OK* in the VIO wizard, Vivado writes an `.xci` file — the IP configuration. *Generate Output Products* then turns that configuration into real files:

Artefact	What it is and who consumes it
<code>vio_0.xci</code>	The IP definition — every parameter you set (probe counts, widths, initial values). This is the file to keep in version control ; everything else is regenerable.
<code>vio_0_stub.v</code>	An empty module with the right port list. Lets the top level elaborate and synthesize while the real contents are compiled separately.
<code>vio_0.veo</code> / <code>.vho</code>	A copy-paste instantiation template. This is where the exact port names in section 4's instantiation came from.
<code>vio_0.dcp</code>	The synthesized checkpoint. By default IP is synthesized Out-Of-Context (OOC) — as its own little run, before and independently of your top-level synthesis.
<code>vio_0_ooc.xdc</code>	Constraints scoped to the IP itself — its internal clocking and any false paths it needs. You never edit this.

The binding is **by module name**, and that is the whole trick. Your HDL says `vio_0 u_vio (...)`; the IP produced a module called `vio_0`; the elaborator matches them. If you rename the IP component in the wizard to anything other than `vio_0`, the instantiation in `vio_debug_top.sv` becomes a **black box** and the design will synthesize with an "unresolved module" warning and then behave as if the VIO were not there.

THE SINGLE MOST COMMON PATH A FAILURE

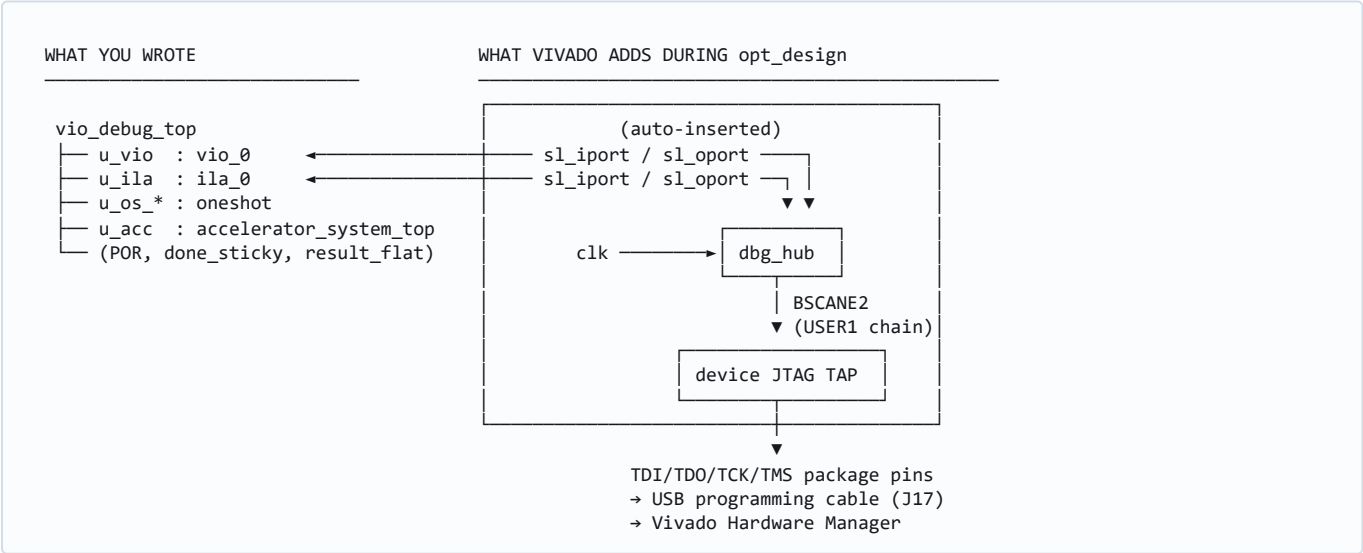
The component names in the wizard must be exactly `vio_0` and `ila_0`, because that is what `vio_debug_top.sv` instantiates. If the Sources tree shows those instances with a black-box icon, or synthesis warns about an unresolved black box, the IP was never generated or was generated under a different name.

DO NOT PROCEED TO IMPLEMENTATION

— the build will complete and the dashboards will simply be absent.

8A.3 What is genuinely automatic — the debug hub

Search `vio_debug_top.sv` for a JTAG port, a TAP controller, or anything named `dbg_hub`. There is nothing. The module has **one port: `clk`**. And yet the Hardware Manager talks to both cores over the programming cable. Here is the missing piece:



Mechanism	What happens, and why you never see it in your source
The hub interface on each core	Every Xilinx debug core carries a dedicated hub-side interface (the <code>sl_iport</code> / <code>sl_oport</code> ports) in addition to the <code>probe*</code> ports you connected. The IP wizard does not show it and the instantiation template omits it, because you are not meant to connect it — the tool does.
Hub insertion	During <code>opt_design</code> , Vivado scans the netlist for debug cores whose hub interface is unconnected. If it finds any and there is no hub yet, it instantiates <code>dbg_hub</code> and wires every core to it . This is why the hub appears in the implemented netlist but in no source file and in no Sources-tree entry.
The JTAG connection	The hub instantiates a BSCANE2 primitive — the fabric's tap into the device's boundary-scan chain — on user scan chain 1 by default (<code>C_USER_SCAN_CHAIN</code>). No package pins are consumed: the JTAG pins are dedicated configuration pins that exist whether you use them or not. This is precisely why a VIO/ILA design needs only one user pin.
Clock selection	The hub needs a fabric clock to run its side of the transfer. Vivado picks the clock of the debug cores it found. When there is any doubt — and there often is — you pin it down explicitly: <code>connect_debug_port dbg_hub/clk [get_nets clk]</code> , exactly as <code>vio_debug.xdc</code> does.
Frequency declaration	The hub must know how fast that clock runs so it can size the JTAG-to-fabric handshake. It does not infer this from <code>create_clock</code> . You tell it: <code>C_CLK_INPUT_FREQ_HZ 100000000</code> and <code>C_ENABLE_CLK_DIVIDER false</code> . Getting this wrong is the number-one cause of " <i>The debug hub core was not detected</i> " — see section 16.
Probe map export	Only the hardware knows probe <i>indices</i> ; only the netlist knows net <i>names</i> . At bitstream time Vivado writes the mapping to a <code>.ltx</code> file. Without it, Hardware Manager sees anonymous cores and can label nothing.

ONE HUB, ONE SCAN CHAIN

A design has

EXACTLY ONE `DBG_HUB`

serving all debug cores, however many you instantiate. If you ever add a JTAG-to-AXI master or a second debug infrastructure, they must sit on *different* user scan chains (`C_USER_SCAN_CHAIN` 1 vs 2 vs 3), or they will fight over the same BSCAN resource and neither will enumerate.

8A.4 Path B in detail — what *Set Up Debug* writes for you

When you right-click a net → *Mark Debug* and run the wizard, Vivado does not edit your RTL. It appends Tcl to a constraints file. The generated commands look like this, and reading them demystifies the whole flow:

```
create_debug_core u_ila_0 ila                                ;# make an ILA in the netlist
set_property C_DATA_DEPTH 1024 [get_debug_cores u_ila_0]
set_property C_TRIGIN_EN false [get_debug_cores u_ila_0]
set_property ALL_PROBE_SAME_MU true [get_debug_cores u_ila_0]
connect_debug_port u_ila_0/clk [get_nets clk]                ;# the capture clock

create_debug_port u_ila_0 probe                              ;# add a probe port
set_property port_width 8 [get_debug_ports u_ila_0/probe0]
connect_debug_port u_ila_0/probe0 [get_nets {u_acc/agu_w_addr[0] u_acc/agu_w_addr[1]}]
```

Command	Effect
<code>create_debug_core</code>	Creates the core object. It does not exist in any HDL file — it is a netlist object.
<code>create_debug_port</code>	Adds a probe port to that core and sets its width.
<code>connect_debug_port</code>	Attaches the port to real nets, by netlist name . This is why <code>-flatten_hierarchy none</code> matters (section 8): flattening renames or dissolves the hierarchical nets you are trying to reference.
<code>set_property MARK_DEBUG true [get_nets ...]</code>	The lighter-weight alternative used in <code>vio_debug.xdc</code> : it just tags nets as debug-worthy and preserves them from optimisation, leaving the core creation to the wizard or to a later run.

WHY MARKING A NET CAN CHANGE YOUR RESULTS — AND WHY IT USUALLY SHOULD NOT

Nets that feed nothing get optimised away. `MARK_DEBUG` forces them to survive, which can change how the surrounding logic is mapped. That is a deliberate, accepted cost of observability — but it means

A DESIGN WITH DEBUG MARKS IS NOT BIT-IDENTICAL TO THE SAME DESIGN WITHOUT THEM

. Always re-run the timing check after adding probes rather than assuming the previous result still holds.

8A.5 The build pipeline, stage by stage

This is what actually happens between clicking *Run Synthesis* and holding a working `.bit`. The debug-relevant events are called out at each stage.

#	Stage	Tcl command	What it does — and what happens to the debug cores
0	IP generation	<code>generate_target / synth_ip</code>	Runs before and independently of your design. Each IP is synthesized out-of-context into its own <code>.dcp</code> . Consequence: changing your RTL does not re-synthesize the VIO , and re-configuring the VIO does not re-synthesize your RTL. They meet later.
1	Elaboration	<code>synth_design -rtl</code>	Builds the RTL-level hierarchy. <code>u_vio</code> and <code>u_ila</code> resolve to the IP stubs — correct ports, empty insides. This is the first place a name mismatch shows up.
2	Synthesis	<code>synth_design</code>	Maps your logic to LUTs, flip-flops, carry chains, BRAM and DSP. The IP instances remain black boxes . Any <code>MARK_DEBUG</code> properties are applied here, which is why marked nets survive optimisation. Output: a post-synthesis checkpoint. Utilisation reported here does <i>not</i> yet include the debug cores.
3	Link / open the design	<code>link_design</code>	The first step of implementation. Vivado stitches the IP .dcp files into the black-box holes . From this moment the VIO and ILA are real gates in the netlist. This is the "meeting point" of the two independent synthesis runs.
4	Optimisation	<code>opt_design</code>	The stage that matters most here. Logic optimisation runs, and — via <code>implement_debug_core</code> — Vivado inserts <code>dbg_hub</code>, connects every debug core to it, instantiates the <code>BSCANE2</code>, and applies your <code>C_CLK_INPUT_FREQ_HZ</code> / <code>C_USER_SCAN_CHAIN</code> properties . If your XDC's <code>get_debug_cores dbg_hub</code> block runs at the right moment, this is when it takes effect.
5	Placement	<code>place_design</code>	Every cell gets a physical site — including the ILA's capture BRAMs and the hub. The debug cores are now competing for the same silicon and the same routing channels as the logic they observe. This is the mechanism behind the timing cost discussed in section 15.
6	Physical optimisation	<code>phys_opt_design</code>	Post-placement timing fixes: replication of high-fan-out drivers, critical-cell re-placement. Optional, and worth enabling if the debug build is tight on slack.
7	Routing	<code>route_design</code>	Real wires. The first point at which timing numbers are sign-off quality — everything before this is an estimate. Hold violations are fixed here automatically.
8	Reports	<code>report_timing_summary</code> , <code>report_utilization</code> , <code>report_drc</code>	Run on the routed design. WNS, WHS and WPWS must all be positive with 0 failing endpoints (section 15, and <code>timing.pdf</code> for the full treatment).
9	Bitstream	<code>write_bitstream</code>	Produces <code>vio_debug_top.bit</code> — and , because debug cores are present, also writes the probes file <code>vio_debug_top.ltx</code> (<code>write_debug_probes</code>). Both land in <code><project>.runs/impl_1/</code> .
10	Program	<code>program_hw_devices</code>	Configures the PL. Hardware Manager reads the <code>.ltx</code> , discovers the hub, enumerates the cores, and creates <code>hw_vio_1</code> and <code>hw_ila_1</code> with your net names attached to the probes.

THE TWO FACTS WORTH MEMORISING FROM THAT TABLE

(1) THE DEBUG HUB IS CREATED AT `OPT_DESIGN`, NOT IN YOUR SOURCE AND NOT AT SYNTHESIS.

That is why searching the RTL for it finds nothing, why post-synthesis utilisation does not show it, and why debug-hub properties must be applied to the implemented design rather than to the elaborated one.

(2) THE `.BIT` AND THE `.LTX` ARE A MATCHED PAIR.

The bitstream carries the hardware; the `.ltx` carries the names. Program with a stale `.ltx` and you get dashboards with wrong or missing probe labels; program without one and you get no dashboards at all, even though the cores are physically present and running.

8A.6 Where everything lands on disk

```
<project>/
├── <project>.srcs/sources_1/
│   ├── imports/                ← the 13 RTL files + vio_debug_top.sv
│   └── ip/vio_0/vio_0.xci, ip/ila_0/ila_0.xci    ← IP configuration (version-control THIS)
├── <project>.srcs/constrs_1/imports/vio_debug.xdc
├── <project>.gen/sources_1/ip/vio_0/            ← generated: stub, .veo template, _ooc.xdc
├── <project>.runs/vio_0_synth_1/vio_0.dcp       ← the OOC synthesized IP
├── <project>.runs/synth_1/vio_debug_top.dcp    ← post-synthesis (IP still black boxes)
├── <project>.runs/impl_1/
│   ├── vio_debug_top_routed.dcp                ← post-route: the sign-off checkpoint
│   ├── vio_debug_top.bit                      ← the bitstream
│   ├── vio_debug_top.ltx                      ← THE PROBES FILE – Program Device needs this
│   └── *_timing_summary_routed.rpt             ← WNS / WHS / WPWS live here
```

8A.7 Verifying the wiring at each stage

After this stage	Run this	You should see
IP generation	Sources tree, IP Sources view	<code>vio_0</code> and <code>ila_0</code> present, not shown as black boxes.
Synthesis	<code>get_cells -hier -filter {REF_NAME =~ *vio*}</code>	The VIO instance exists in the netlist.
Synthesis	<code>get_cells -hier u_os_start</code>	The <code>oneshot</code> instances survived — critical, see section 4.
Synthesis	<code>get_nets -hier -filter {MARK_DEBUG}</code>	Every net you marked in the XDC, and nothing you did not.
<code>opt_design</code> / open implemented design	<code>get_debug_cores</code>	<code>dbg_hub</code> , <code>u_ila</code> , <code>u_vio</code> — this is the proof the automatic hub insertion happened.
Implemented design	<code>get_property C_CLK_INPUT_FREQ_HZ [get_debug_cores dbg_hub]</code>	<code>100000000</code> . If it reads something else, your XDC block did not run.
Implemented design	<code>report_timing_summary</code>	WNS, WHS, WPWS positive; 0 failing endpoints.
Bitstream	File listing of <code>impl_1/</code>	Both <code>.bit</code> and <code>.ltx</code> present, with the same timestamp.
Programming	Hardware pane	<code>hw_vio_1</code> and <code>hw_ila_1</code> under <code>xc7z020_1</code> .
Connected	<code>accel_probes</code> (section 12)	17 probes with the widths of sections 6 and 7 — 6 IN, 11 OUT.

8A.8 The same build without the GUI

Every stage above is a Tcl command, which is what makes the whole flow reproducible. A minimal non-project script for this design:

```

create_project -in_memory -part xc7z020clg484-1

read_verilog -sv [glob <RTL_DIR>/*.sv]
read_verilog -sv FPGA_IMPL/rtl/vio_debug_top.sv
read_ip      vio_0/vio_0.xci
read_ip      ila_0/ila_0.xci
synth_ip     [get_ips]                ;# stage 0 – OOC synthesis of the debug cores
read_xdc     FPGA_IMPL/vio_debug.xdc

synth_design -top vio_debug_top -part xc7z020clg484-1 ;# stages 1-2
opt_design   ;# stages 3-4 ← dbg_hub inserted HERE
place_design ;# stage 5
phys_opt_design ;# stage 6
route_design ;# stage 7

report_timing_summary -file timing_summary_routed.rpt ;# stage 8 – sign-off numbers
report_utilization   -file utilization_routed.rpt
report_drc           -file drc_routed.rpt

write_bitstream -force vio_debug_top.bit ;# stage 9
write_debug_probes -force vio_debug_top.ltx ;# the probes file, explicitly

```

SANITY-CHECK YOU CAN RUN IN TEN SECONDS

Open the implemented design and type `get_debug_cores`. If the answer includes `dbg_hub`, the automatic wiring worked and any remaining problem is on the host side — cable, drivers, `.ltx`, or JTAG frequency. If `dbg_hub` is absent, the problem is in the build, and the usual cause is that the IP was a black box all along (section 8A.2).

9. Constraints, synthesis, implementation, bitstream

9.1 The constraints file

`vio_debug.xdc` is short because there is exactly one physical pin:

```

set_property -dict {PACKAGE_PIN Y9 IOSTANDARD LVCMOS33} [get_ports clk]
create_clock -name sys_clk -period 10.000 [get_ports clk]

set_property BITSTREAM.GENERAL.UNCONSTRAINEDPINS {Allow} [current_design]

if {[llength [get_debug_cores -quiet dbg_hub]]} {
    set_property C_CLK_INPUT_FREQ_HZ 100000000 [get_debug_cores dbg_hub]
    set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
    set_property C_USER_SCAN_CHAIN 1 [get_debug_cores dbg_hub]
    connect_debug_port dbg_hub/clk [get_nets clk]
}

```

VERIFY THE CLOCK PIN AGAINST THE DILIGENT MASTER XDC

Y9 is the 100 MHz PL oscillator on the widely used ZedBoard rev C/D master constraints. Board revisions differ. Download the official `zedboard_master.xdc` for *your* revision and confirm the `PACKAGE_PIN` and `IOSTANDARD` before you build. A wrong clock pin produces a bitstream that programs successfully and then does absolutely nothing — the hardest kind of failure to diagnose.

THE DEBUG-HUB CLOCK BLOCK IS THE SINGLE MOST VALUABLE CONSTRAINT HERE

Vivado auto-inserts a `dbg_hub` to bridge JTAG to your VIO and ILA. If it does not know the clock frequency it guesses, and the Hardware Manager then reports *"The debug hub core was not detected"* — you get a programmed device with no dashboards at all. Setting `C_CLK_INPUT_FREQ_HZ` to the real 100 MHz and disabling the clock divider fixes the overwhelming majority of these cases.

9.2 Build

1. **Run Synthesis** (*Flow Navigator* → *Run Synthesis* → OK). Expect zero errors. Warnings about the unconnected `btn1/btnr` -style ports do not apply here — `vio_debug_top` has one port.
2. **Run Implementation** → OK.
3. **Check timing.** *Reports* → *Timing* → *Report Timing Summary*. **WNS must be positive.** For reference, the accelerator alone synthesizes to **WNS +1.279 ns** at 100 MHz (\$15) — a working margin, but not a huge one, and the debug cores you are adding will eat into it. Check your `create_clock` first because that is cheap to verify, but treat a negative WNS as a real result rather than assuming it must be a constraint typo.
4. **Generate Bitstream** → OK.

IF TIMING FAILS

You have an easy escape here that most designs do not:

NONE OF THE FOUR INTERFACE METHODS DEPENDS ON THROUGHPUT.

Add an MMCM and clock the fabric at 50 MHz, which doubles the available period and makes the problem disappear. Remember to update `C_CLK_INPUT_FREQ_HZ` on the debug hub to match, or you will trade a timing failure for a "debug hub not detected". Reducing the ILA sample depth also helps, since it frees both block RAM and the routing around it.

10. Programming the board and opening the dashboards

1. Power the board, connect the micro-USB to **J17**.
2. *Flow Navigator* → **Open Hardware Manager** → **Open Target** → **Auto Connect**.
3. The *Hardware* pane shows the chain: `localhost` → `xilinx_tcf` → `arm_dap_0` and `xc7z020_1`.
4. Right-click `xc7z020_1` → **Program Device**. The `.bit` path and the matching `.ltx` *debug probes file* are pre-filled — **both must be present**. Click **Program**.
5. After a few seconds the Hardware pane grows two children under the device: `hw_vio_1` and `hw_ila_1` (plus `hw_ila_2` if you also used the Set Up Debug wizard). Double-click each to open its dashboard.

THE .LTX FILE IS NOT OPTIONAL

The probes file maps probe indices to signal names. Without it the dashboards show anonymous `probe_out4` -style entries, or refuse to open. Vivado writes it next to the bitstream during implementation. If the field is blank, browse to `<project>.runs/impl_1/vio_debug_top.ltx`.

10.1 Setting up the VIO dashboard

1. In the `hw_vio_1` window, click the + in the *VIO Probes* pane → **Select All** → OK.
2. For each 32-bit and 128-bit probe, right-click → *Radix* → **Hex**. For `done_count` choose **Unsigned Decimal**.
3. For the output probes you will type into (`vio_wdata`, `vio_adata`, `vio_waddr`, `vio_aaddr`, `vio_oaddr`, `vio_num_vectors`), the **Value** column becomes an editable text box.
4. For the 1-bit command probes (`vio_start_lvl`, `vio_w_we_lvl`, `vio_a_we_lvl`), right-click → **Toggle Button** so you get a clickable switch.
5. Confirm the refresh mode: the small circular-arrow icon should show *auto-refresh*, or click it manually before each read.

SAVE THE DASHBOARD

Once the probes, radices and toggle buttons are arranged, use *File* → *Save Constraints* (or export the dashboard) so this layout is restored the next time you open the Hardware Manager. Rebuilding it by hand every session is tedious and error-prone.

11. Running the demo from the VIO dashboard (manual)

This is the T5 golden case — the same vectors the simulation regression checks, so a mismatch here is unambiguously a hardware or setup problem, not a modelling question.

$$\begin{array}{c} \text{A (4x4)} \\ \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{array} \times \begin{array}{c} \text{W (4x4)} \\ \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \end{array} = \begin{array}{c} \text{C = A x W (4x4)} \\ \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 28 & 32 & 36 & 40 \end{bmatrix} \end{array}$$

11.1 Step 1 — confirm the accelerator is idle

On the dashboard: `busy` must read **0**. If it does not, the reset never released or a command bit is stuck high — see section 16.

11.2 Step 2 — write the four weight words

For each row: type `vio_w_wdata`, type `vio_w_addr`, then click `vio_w_we_lv1` to **1** and back to **0**. The 0→1 edge produces exactly one clock of `host_w_we`; returning to 0 re-arms the one-shot for the next write.

<code>vio_w_addr</code>	<code>vio_w_wdata</code>	Weight row	Byte breakdown (lane 0 = low byte = column 0)
0	0x04030201	W[0] = 1 2 3 4	04 = W[0][3] 03 = W[0][2] 02 = W[0][1] 01 = W[0][0]
1	0x08070605	W[1] = 5 6 7 8	08 07 06 05
2	0x0C0B0A09	W[2] = 9 10 11 12	0C 0B 0A 09
3	0x100F0E0D	W[3] = 13 14 15 16	10 0F 0E 0D

WRITE THE ROWS IN NATURAL ORDER 0, 1, 2, 3

Do *not* reverse them. The bottom-first shift into the array is performed by the AGU during WLOAD (it counts `w_addr` down 3, 2, 1, 0). Reversing them here produces a wrong answer that still looks structurally plausible.

11.3 Step 3 — write the four activation words

Same procedure with `vio_a_wdata`, `vio_a_addr`, `vio_a_we_lv1`.

<code>vio_a_addr</code>	<code>vio_a_wdata</code>	Activation row	Byte breakdown (lane 0 = low byte = array row 0)
0	0x00000001	A[0] = 1 0 0 0	00 = A[0][3] 00 00 01 = A[0][0]
1	0x00000100	A[1] = 0 1 0 0	00 00 01 00
2	0x00010000	A[2] = 0 0 1 0	00 01 00 00
3	0x01010101	A[3] = 1 1 1 1	01 01 01 01

11.4 Step 4 — configure and launch

- `vio_num_vectors` = **4** (already the initial value if you followed section 6.3)
- `vio_act_base` = **0**, `vio_out_base` = **0**
- Note the current `done_count`.
- Click `vio_start_lv1` to **1**, then back to **0**.

The job runs in **17 clocks = 170 ns**. It is over long before the next JTAG transaction lands.

11.5 Step 5 — confirm it ran

- done_sticky = 1
- done_count incremented by **exactly 1** — this is what proves the one-shot works. If it jumped by thousands, vio_start_lvl is being held high or the oneshot was optimized away.
- busy = 0 again

11.6 Step 6 — read the results

Set vio_o_addr to 0, 1, 2, 3 in turn and read o_rdata. The SRAM has one clock of read latency; a JTAG round trip is six orders of magnitude longer, so no wait is needed.

vio_o_addr	o_rdata (128-bit hex)	Decodes to C row
0	0000_0004_0000_0003_0000_0002_0000_0001	[1 2 3 4]
1	0000_0008_0000_0007_0000_0006_0000_0005	[5 6 7 8]
2	0000_000C_0000_000B_0000_000A_0000_0009	[9 10 11 12]
3	0000_0028_0000_0024_0000_0020_0000_001C	[28 32 36 40]

Read the 128-bit word right to left: bits [31:0] are column 0. Row 3 is the interesting one — the all-ones activation row yields the column sums 28 32 36 40, and 0x1C 0x20 0x24 0x28 = 28 32 36 40.

11.7 result_flat reads all zeros — this is correct, not a fault

DO NOT READ RESULTS FROM RESULT_FLAT ON THE VIO DASHBOARD. IT WILL ALWAYS BE 0.

This is the single most common confusion with this harness, and it is a property of the architecture, not a bug in the design, the harness or the probe.

There are two different result paths, and only one of them is observable from VIO:

Probe	What it is	Valid when	Readable from VIO?
o_rdata probe_in3	Stored result — a registered read port on output_mem	Persistently , any time after the job, for as long as you leave the memory alone	YES — this is where you read results. Set vio_o_addr, read o_rdata.
result_flat probe_in4	Live combinational bus straight off the de-skew buffer, ahead of the output SRAM	For four clocks total — samples 13–16 of a 17-clock job	NO. Use the ILA (probe8).

Why it is exactly zero rather than stale garbage

Walk the timeline. The four result vectors cross result_flat during DRAIN and are captured into output_mem by the AGU's write strobes. After the last one, the array keeps draining: valid_in is 0, so each PE takes the pass-through path (psum_q <= psum_in) rather than accumulating, and the north edge is driven with zeros because wload_q is 0. Those zeros march south one row per clock, clearing the weight residue out of the psum_q registers as the first activations enter behind them. Within a few cycles of DONE the entire fabric — and therefore result_flat — has settled to 0, and it stays there until the next job.

Now compare the timescales:

```
window in which result_flat carries data : 4 clocks = 40 ns
whole job                               : 17 clocks = 170 ns
one JTAG round trip to refresh the VIO  :           ~ 1-10 ms

ratio ~ 1 : 100 000+
```

VIO samples when *you* click refresh. The probability of that landing inside a 40 ns window is effectively zero, and even if it did, VIO has no trigger — it cannot wait for the interesting moment. So the dashboard shows the settled value, which is 0. **A VIO probe on a live datapath bus is the wrong instrument.** That is precisely the division of labour between the two cores: VIO is for *state you can take your time over*, the ILA is for *events*.

SO WHY IS RESULT_FLAT WIRED TO THE VIO AT ALL?

Honestly, mostly for symmetry with the ILA probe list and as a sanity anchor — an all-zero reading confirms the fabric drained cleanly, which is a real (if weak) health signal. If you find it more confusing than useful, delete `probe_in4` from the VIO instantiation and drop *Input Probe Count* from 6 to 5. Nothing else in this document depends on it. Keep it on the

ILA

(probe8) — there it is one of the most valuable signals you have, because that is where you watch C0...C3 emerge one per clock (§13.3).

THE SAME REASONING APPLIES TO PHASE AND BUSY

phase will read

0 (IDLE)

and busy will read

0

every single time you look, for the same reason — the job is over 170 ns after you launch it. They are on the VIO so you can confirm the accelerator is *idle and ready* before loading, not so you can watch a job progress. To watch a job progress you need the ILA. The probes that *are* meaningful on the VIO after a run are `done_sticky` and `done_count`, and they are meaningful precisely because the harness added hardware to

LATCH

transient events into persistent state.

12. Running the demo from Tcl (scripted)

Twenty-four dashboard clicks per run gets old fast, and a mis-click is indistinguishable from a hardware bug.

`FPGA_IMPL/vio_demo.tcl` does the whole sequence and checks all sixteen results against the golden model.

```
# in the Hardware Manager Tcl console, AFTER Program Device
cd C:/Users/VLSI/Desktop/systolic_array_multiplier/FPGA_IMPL
source vio_demo.tcl
accel_demo
```

Expected output:

```
=== 4x4 systolic accelerator :: VIO demo (T5 golden case) ===
loading weights...
loading activations...
running (num_vectors = 4, 17 clocks = 170 ns)...
job done (done_count 1 , was 0)

C = A x W :
[ 1 2 3 4 ] OK
[ 5 6 7 8 ] OK
[ 9 10 11 12 ] OK
[ 28 32 36 40 ] OK

PASS -- hardware matches the simulation golden model.
```

12.1 The Tcl primitives it is built from

Command	Effect
<code>get_hw_vios</code>	Returns the VIO core objects on the connected device
<code>get_hw_probes -of_objects \$vio</code>	Returns every probe; filter with <code>-filter {NAME =~ *pattern*}</code>
<code>set_property OUTPUT_VALUE_RADIX HEX \$p</code>	Sets how <code>OUTPUT_VALUE</code> is interpreted
<code>set_property OUTPUT_VALUE 04030201 \$p</code>	Stages a new value locally — does not reach the device yet
<code>commit_hw_vio \$p</code>	Pushes the staged value over JTAG. Forgetting this is the #1 scripting mistake.
<code>refresh_hw_vio \$vio</code>	Pulls current <code>probe_in</code> values from the device
<code>get_property INPUT_VALUE \$p</code>	Reads the cached snapshot — meaningless without a preceding refresh

PROBE NAMES VARY BETWEEN VIVADO VERSIONS

Vivado names each `hw_probe` after the net attached to the VIO port, but the exact string differs by version — `vio_w_wdata` in some, a hierarchical `vio_debug_top_i/vio_w_wdata` in others. The script therefore resolves a probe by **EXACT MATCH ON THE LAST PATH COMPONENT FIRST**

, falling back to a substring match, and

ALWAYS FILTERING BY DIRECTION

so a value or radix call can never land on the wrong probe. On ambiguity it lists the candidates instead of silently picking one. Run `accel_probes` at any time to print the actual names, widths and directions.

12.2 Individual entry points

```
accel_probes          # list probe names / widths / directions
accel_write_w 0 04030201 # one weight word (2-bit addr, hex data)
accel_write_a 3 01010101 # one activation word (8-bit addr, hex data)
accel_run 4             # num_vectors=4, poll done_sticky
accel_read 3           # -> {28 32 36 40}, col0 first, sign-extended int32
accel_demo             # the whole T5 sequence with pass/fail checking
```

12.3 Sweeping — what scripting really unlocks

The design supports up to **256** activation vectors against a single weight load (that amortisation is the entire economic point of a weight-stationary array). Loading 256 vectors by hand is not realistic; from Tcl it is a loop:

```
for {set i 0} {$i < 64} {incr i} {
    accel_write_a $i [format %08X [expr {(($i & 0xFF) * 0x01010101)}]]
}
accel_run 64 ;# 64 + 13 = 77 clocks = 770 ns
for {set i 0} {$i < 64} {incr i} { puts "row $i : [accel_read $i]" }
```

This is the one place the 8-bit `vio_a_addr` earns its width — see the box in §6.3.

12.4 If the script errors — what each failure means

CORRECTED IN THIS EDITION: A REAL LANE-ORDERING BUG

The first edition's `accel_read` unpacked the 128-bit word into `{col3 col2 col1 col0}` while `accel_demo` compared against a golden `{col0 col1 col2 col3}`. The result was

MISMATCH REPORTED ON ALL FOUR ROWS EVEN WHEN THE HARDWARE WAS PERFECTLY CORRECT

— you would have seen `[4 3 2 1]` where the golden says `[1 2 3 4]`. If your output was a

REVERSED BUT OTHERWISE CORRECT

matrix, that was this bug and not your board. Fixed by walking the hex chunks from the end backwards.

Error / symptom	Cause	Fix
No hardware target open	The script was sourced before connecting	Open Hardware Manager → Open Target → Auto Connect first, then <code>source vio_demo.tcl</code> .
No VIO core found	Bitstream not programmed, <code>.ltx</code> missing, or the wrong device in the chain was programmed	Program <code>xc7z020_1</code> , not <code>arm_dap_0</code> . Confirm the <i>Debug probes file</i> field pointed at <code>vio_debug_top.ltx</code> . Confirm <code>hw_vio_1</code> is visible in the Hardware pane.
No OUTPUT probe matching 'vio_start_lv1'	Your Vivado named the probes differently, or the harness was edited	Run <code>accel_probes</code> and use the names it prints. The lookup already tries exact-tail then substring, so this means the string genuinely is not there.
Ambiguous OUTPUT probe ...	Two probes share the substring	Pass a longer, more specific name — the message lists every candidate.
ERROR: [Labtools 27-*] ... not a valid property on OUTPUT_VALUE	The resolved probe is an input ; inputs have no <code>OUTPUT_VALUE</code>	Fixed in this edition — every lookup now filters by <code>DIRECTION</code> . If you adapted the script, keep that filter.
Values written but nothing happens	<code>set_property</code> without <code>commit_hw_vio</code>	<code>set_property</code> only stages locally. commit_hw_vio is what drives JTAG . The #1 scripting mistake.
Reads return the same stale value forever	<code>get_property INPUT_VALUE</code> without <code>refresh_hw_vio</code>	<code>INPUT_VALUE</code> is a cached snapshot. Refresh pulls from the device first.
<code>accel_run</code> times out	<code>start</code> never pulsed, or the job never completed	Check <code>done_count</code> before and after. If it did not move, the one-shot never fired — verify <code>vio_start_lv1</code> actually toggled 0→1→0 and that <code>busy</code> was 0 beforehand.
Results are a reversed matrix	The lane-order bug above	Use this edition's <code>accel_read</code> .
Results are all 0	Host writes were masked, or you read <code>result_flat</code> instead of <code>o_rdata</code>	See §11.7. Read via <code>vio_o_addr</code> → <code>o_rdata</code> .
Short values decode wrong	Vivado drops leading zeros in <code>INPUT_VALUE</code>	Fixed — <code>accel_read</code> now left-pads to 32 hex characters before slicing.

DEBUG THE SCRIPT ONE LAYER AT A TIME

When something misbehaves, work bottom-up rather than re-running `accel_demo`:

```

accel_probes                ;# 1. are the probes there, right widths, right directions?
accel_get busy BINARY       ;# 2. can I read at all? should be 0
accel_get done_count        ;# 3. note the value
accel_write_w 0 04030201    ;# 4. can I write?
accel_run 4                 ;# 5. did done_count increment by exactly 1?
accel_read 0                ;# 6. does the unpacking look sane?

```

13. Reading the ILA waveform — the expected cycle map

This is what Method 2 exists for. The design has **fixed, derivable latency and no back-pressure anywhere**, so every number below is a hard constant you can check the silicon against.

13.1 Arm and capture

1. Open the **hw_ila_1** dashboard.
2. In *Trigger Setup*, click + and add `probe0 (start_p)`.
3. Set the condition to `== 1`.
4. In *Settings*, set **Trigger position in window** to **64** (so you capture 64 samples of context before the launch — enough to see the last host write land).
5. Click **Run Trigger** (▶). Status goes to *Waiting for Trigger*.
6. Cause a run: click `vio_start_lv1 0→1→0` on the VIO dashboard, or call `accel_run 4` from Tcl.
7. The waveform fills instantly.

13.2 The cycle map for a four-vector job

Let **sample 0** be the sample in which `start_p` is high. With `num_vectors = 4`:

Sample	phase	busy	Cycles	What is happening
0	0 IDLE	0	1	<code>start_p = 1</code> . The FSM samples it (it is only sampled in IDLE).
1 – 4	1 WLOAD	1	4 = ROWS	Weight rows shifted into the array's north edge, bottom-first : the AGU drives <code>w_addr = 3, 2, 1, 0</code> with <code>wload = 1</code> .
5 – 8	2 COMPUTE	1	4 = <code>num_vectors</code>	One dense activation vector per clock, <code>valid_in = 1</code> . <code>act_addr</code> walks <code>act_base + 0...3</code> . COMPUTE follows WLOAD directly — there is no FLUSH phase.
9 – 16	3 DRAIN	1	8 = STORE_LATENCY	The last vector is still in flight through the array and the de-skew pipe. The four result writes land at samples 13–16; the weight residue left by WLOAD clears the result bus at sample 12.
17	4 DONE	0	1	<code>done</code> pulses for exactly one clock. Note <code>busy</code> is already low .
18 →	0 IDLE	0	—	<code>done_count</code> has incremented by 1; <code>done_sticky</code> is set.
Weight reuse: if no <code>host_w_we</code> has occurred since the previous job, WLOAD is skipped entirely – samples 1–4 are COMPUTE, DRAIN runs 5–12, DONE at 13. The ILA is the easiest place to see this: the <code>phase</code> probe goes 0 → 2 with no 1 in between.				

Total = 4 + 4 + 8 + 1 = 17 = `num_vectors` + 13 (13 clocks when the weight load is skipped). Counting the phase-enable widths on the waveform and getting 4 / N / 8 / 1 is the strongest single confirmation that the hardware matches the specification.

13.3 Result checkpoints

Two derived latencies govern when data appears. Both are measured from the cycle in which the activation *address* is issued:

Constant	Value	Derivation	Observable as
L1 result latency	6	ROWS+COLS-2 (3 down a column, +c east, +(COLS-1-c) de-skew — the c terms cancel)	Independent of column: all four lanes align on the same clock.
L2 result latency	7	1 (a_mem read) + 6	result_flat (probe8) shows C _i around sample 18+i → samples 18, 19, 20, 21 .
STORE_LATENCY	8	result latency + 1 (strobe one cycle after data is observable)	out_we fires around samples 19, 20, 21, 22 — all four inside DRAIN, which is exactly why DRAIN is 8 cycles long.

On result_flat (probe8) you should see, one per clock:

```
C0 = 0000_0004 0000_0003 0000_0002 0000_0001 -> [ 1 2 3 4 ]
C1 = 0000_0008 0000_0007 0000_0006 0000_0005 -> [ 5 6 7 8 ]
C2 = 0000_000C 0000_000B 0000_000A 0000_0009 -> [ 9 10 11 12 ]
C3 = 0000_0028 0000_0024 0000_0020 0000_001C -> [ 28 32 36 40 ]
```

ALLOW ±1 SAMPLE WHEN MATCHING THESE AGAINST THE ILA

The design documentation distinguishes carefully between a value *issued* at cycle *n* (the value the *n*-th edge samples) and *observed* at cycle *n* (the value just after that edge). Conflating the two caused two of the three historical defects found during verification. The ILA registers its probes, so a given sample index may sit one clock either side of the "issued" convention used in the table.

PHASE LENGTHS (4/6/N/8/1) ARE EXACT AND UNAMBIGUOUS — ANCHOR ON THOSE.

Treat the result-checkpoint sample numbers as ±1 and check the *spacing* (one clock apart, four in a row) rather than the absolute index.

13.4 What a healthy capture looks like

- start_p — a single 1-clock spike, never a plateau.
- busy — high for exactly 22 samples, then low before done fires.
- phase — the staircase 0→1→2→3→4→5→0 with widths 1/4/6/4/8/1.
- done — one 1-clock spike at sample 17 (13 if WLOAD was skipped).
- done_count — increments by exactly 1 within the window.
- result_flat — quiet until the first result lands, then four distinct values on four consecutive clocks late in the run.
- w_we_p / a_we_p — if the trigger window reaches back far enough, single 1-clock spikes only. Any plateau means a one-shot was optimized away.

14. Trigger recipes

Goal	Trigger setup	Notes
Capture a whole job (default)	<code>probe0 (start_p) == 1</code> , trigger position 64	The canonical capture. 64 samples of pre-trigger context show the last host write landing.
Capture the completion	<code>probe2 (done) == 1</code> , trigger position 900	Puts the trigger near the end of the window so almost all 1024 samples are <i>pre-trigger</i> — you see the entire job leading up to <code>done</code> plus the host writes before it.
Catch a specific phase	<code>probe3 (phase) == 3</code> (COMPUTE)	Useful when sweeping <code>num_vectors</code> and you only care about the streaming window.
Catch a wrong answer	Trigger comparator on <code>probe8 (result_flat) != expected</code>	Requires the probe to be wide enough for a comparator; set the compare value in the trigger pane. The killer application for the ILA — freeze the exact cycle where hardware diverges from the golden model.
Watch back-to-back jobs	<code>probe9 (done_count) == 2</code>	Confirms the FSM cleanly accepts a fresh job after the previous one, and that <code>start</code> is genuinely ignored while busy.
Prove <code>num_vectors = 0</code> does not hang	<code>probe0 == 1</code> , with <code>vio_num_vectors</code> set to 0	The FSM must exit WLOAD straight to DRAIN, skipping COMPUTE. Total job = 0 + 13 = 13 clocks (or 9 if the weight load is also skipped). Without the guard, COMPUTE would sit for 256 cycles issuing garbage writes.

14.1 Multi-condition triggers

Change *Trigger Mode* from `BASIC_ONLY` to `BASIC_OR_TRIG_IN` or use the **Advanced Trigger** state machine for sequences. A useful one here: "*trigger when phase enters DRAIN and result_flat is still zero*" catches a de-skew-depth bug, one of the failure modes the design's own coverage notes flag as capable of going silently wrong.

14.2 Trigger position

The *Trigger position in window* field decides how the 1024 samples are split between pre- and post-trigger. Since the whole job is 17 samples, almost any position works; the value only matters when you want to see what happened *before* the trigger.

14.3 Capture control (storage qualification)

If you enabled *Capture Control* when generating the ILA, you can set a storage condition such as `busy == 1`, so the buffer only stores samples during jobs. With a 17-cycle job and a 1024-sample buffer, this lets one capture hold ~44 consecutive jobs back to back — the fastest way to spot an intermittent failure across a sweep.

15. Resource and timing — measured

The figures below are **measured**, not estimated: out-of-context synthesis of `accelerator_system_top` alone (no VIO, no ILA, no debug hub) for `xc7z020c1g484-1` in **Vivado 2020.1**, constrained at 100 MHz. Synthesis completed with **0 errors, 0 warnings and 0 critical warnings**.

Resource	Used	Available	%	Note
Slice LUTs	1 564	53 200	2.94	1 524 as logic + 40 as memory
Slice Registers	792	106 400	0.74	
Block RAM Tile	2.5	140	1.79	output_mem (32 kbit) + a_mem (8 kbit)
DSPs	0	220	0.00	The sixteen 8×8 signed multipliers mapped to fabric logic, not DSP48E1 slices
LUT as Memory	40	17 400	0.23	weight_mem 4×32 as distributed RAM — as predicted, a deeper weight memory would be dead silicon

15.1 Timing — read this carefully

```
Constraint : create_clock -period 10.000 (100 MHz)
WNS  = +1.279 ns      0 failing endpoints of 1305
WHS  = +0.263 ns      0 failing endpoints of 1305
WPWS = +3.750 ns      0 failing endpoints of 838
```

CORRECTION TO THE FIRST EDITION

This document previously described the timing margin as "ample" and "enormous", and advised that a negative WNS should make you suspect the constraint rather than the design.

THAT WAS OVERSTATED.

+1.279 ns on a 10 ns period is a

~13% MARGIN

— comfortable and passing, but not enormous. Two things can erode it:

- These are

POST-SYNTHESIS

numbers. Place-and-route adds real routing delay, and post-synthesis estimates on 7-series are generally optimistic relative to post-route.

- This build contains

NO DEBUG LOGIC

. Adding the VIO, the ILA and the auto-inserted debug hub consumes LUTs, registers, block RAM and — more importantly — routing resources near your logic.

So treat a negative WNS in *your* build as a genuine result worth investigating, not as an automatic sign of a bad constraint. Check the constraint first because it is cheap to check, but do not dismiss the design.

15.1.1 The debug build, measured post-implementation

The figures above are the accelerator *alone*. The build you actually program — the full `vio_debug_top` with the VIO, the ILA and the auto-inserted `dbg_hub` — reports the following Design Timing Summary:

	Setup	Hold	Pulse Width
Worst slack	WNS +0.714 ns	WHS +0.045 ns	WPWS +3.750 ns
Total slack	TNS 0.000 ns	THS 0.000 ns	TPWS 0.000 ns
Failing endpoints	0	0	0
Total endpoints	5 181	5 181	2 950

"All user specified timing constraints are met." — a clean pass, with roughly 7 % margin on the 10 ns period.

The endpoint count rising from 1 305 to 5 181 is the debug infrastructure becoming real flip-flops at `link_design/opt_design` (§8A.5), and the ~0.565 ns of setup slack between the two builds is the price of observability. **Full treatment of setup, hold, slack, WNS and closure is in `timing.pdf`.**

15.2 Cost of the debug cores

Core	Cost	Note
ila_0 at depth 1024	≈ 8 BRAM36	288 probe bits × 1024 = 294 912 bits; BRAM36 = 36 864 bits. This is more than 3× the accelerator's own memory footprint — the analyzer is bigger than the thing it analyses.
ila_0 at depth 256	≈ 2 BRAM36	Recommended. The whole job is 17 clocks; 256 samples is ample and saves 6 BRAM36.
vio_0	Negligible	Registers plus JTAG glue.
dbg_hub	Small, auto-inserted	You do not instantiate it; Vivado adds it when any debug core is present.

REPRODUCE THESE NUMBERS YOURSELF

```
create_project -in_memory -part xc7z020clg484-1
foreach f [glob <RTL_DIR>/*.sv] { read_verilog -sv $f }
read_xdc clk_only.xdc                ;# just: create_clock -period 10.000 [get_ports clk]
synth_design -top accelerator_system_top -part xc7z020clg484-1 -mode out_of_context
report_utilization
report_timing_summary
```

16. Troubleshooting

Symptom	Most likely cause	Fix
"The debug hub core was not detected" in Hardware Manager	The debug hub does not know its clock frequency, or the JTAG clock is too fast relative to the design clock	Confirm the <code>dbg_hub</code> block in <code>vio_debug.xdc</code> ran. Set <code>C_CLK_INPUT_FREQ_HZ 100000000</code> and <code>C_ENABLE_CLK_DIVIDER false</code> . Also try lowering the JTAG frequency: right-click the target → <i>Open Target Properties</i> → reduce <code>Frequency</code> to 6 MHz or less.
No <code>hw_vio_1</code> / <code>hw_ila_1</code> under the device	The <code>.ltx</code> probes file was not loaded with the bitstream	Re-run <i>Program Device</i> and set the <i>Debug probes file</i> field to <code><project>.runs/impl_1/vio_debug_top.ltx</code> .
Device does not appear at all under Auto Connect	Cable drivers, wrong USB port, or boot-mode jumpers	Use J17 (not J14). Confirm <code>PG00D</code> is lit. Set <code>JP7–JP11</code> to JTAG (all GND). Reinstall Vivado's cable drivers.
<code>busy</code> reads 1 forever	Reset never released, or <code>start</code> is stuck high	The POR is 65 536 clocks — it releases in 655 μs, so this is not a wait problem. Check <code>vio_start_lv1</code> is 0 on the dashboard, and confirm the <code>oneshot</code> instances survived synthesis (search the netlist for <code>u_os_start</code>).
<code>done_count</code> jumps by thousands per click	The one-shot was optimized away, or you edited it out	Confirm <code>oneshot</code> is present. Check the elaborated design shows <code>start_p</code> driven by a flop, not directly by the VIO.
<code>done_sticky</code> never sets	<code>num_vectors</code> is fine but the job never launched — <code>start</code> arrived while <code>busy</code> was high, so it was ignored	Read <code>busy</code> first; it must be 0 before you click start. Also confirm <code>done_count</code> did not move.
All results read 0	Host writes were masked because <code>busy</code> was 1, or the writes never pulsed	Re-load with <code>busy == 0</code> . Add the ILA trigger on <code>w_we_p == 1</code> and confirm you see exactly four 1-clock spikes.
Results are a transposed or scrambled matrix	Lane order wrong in the packed word	Lane 0 is the low byte. <code>weight_mem[r]</code> lane <code>c = W[r][c]</code> ; <code>a_mem[i]</code> lane <code>r = A[i][r]</code> . Re-check against Appendix A.
Row 3 reads <code>28 32 36 40</code> but rows 0–2 are wrong	Weight rows written in reverse order	Write <code>weight_mem[0..3]</code> in natural order. The bottom-first shift is the AGU's job. Row 3 (all-ones activation) is the column sum and is insensitive to row permutation — which is exactly why this failure hides there.
Results read as <code>X</code> or garbage in simulation but fine on hardware	The memories have no reset; unwritten addresses return <code>X</code> in simulation	Expected behaviour. Write every address you intend to read.
ILA triggers continuously, waveform unusable	<code>start</code> held high (one-shot bypassed)	Same fix as the <code>done_count</code> row above.
A <code>MARK_DEBUG</code> constraint errors with "no nets matched"	Synthesis renamed or absorbed the net	Set <code>-flatten_hierarchy</code> to <code>none</code> , then discover the real name with <code>get_nets -hierarchical *pattern*</code> in the Synthesized Design.
Bitstream generation fails on unconstrained PS pins	PL-only Zynq DRC	<code>set_property BITSTREAM.GENERAL.UNCONSTRAINEDPINS {Allow} [current_design]</code> — already in <code>vio_debug.xdc</code> .
Elaboration error on <code>vio_0</code> / <code>ila_0</code>	Generated probe widths do not match the instantiation	Re-customize the IP and align every width with the tables in sections 6.2, 6.3 and 7.2.
Syntax errors on <code>logic</code> / <code>always_ff</code>	Files added as Verilog, not SystemVerilog	Right-click each source → <i>Set File Type</i> → <i>SystemVerilog</i> .

17. Extending the harness

Extension	How
Long streaming jobs	<code>a_mem</code> holds 256 vectors against a single weight load. Script the loads (section 12.3) and set <code>num_vectors</code> up to 255. Once the pipe is full it is one vector per clock — measure it with an ILA capture over <code>busy</code> .
<code>act_base</code> / <code>out_base</code> isolation	Load two independent activation blocks at different bases, run two jobs with different <code>out_base</code> , and confirm neither disturbs the other. This mirrors a check the simulation regression already performs.
Adversarial arbitration test	Hold <code>vio_a_we_lv1</code> and <code>vio_w_we_lv1</code> toggling with garbage data while a job runs, and confirm the results are bit-identical. Port arbitration should mask every one of them. The simulation regression does this with <code>0xDEADBEEF</code> / <code>0xBAADF00D</code> .
Add the array internals to the ILA	Section 8. Probing <code>psum_out_bottom</code> lets you literally watch the diagonal wavefront cross the array on silicon.
Combine with Method 1	Build <code>fpga_top</code> (switches + LEDs) as the top and add ILA/VIO to it. You then get the standalone board demo <i>and</i> cycle-accurate visibility of the same run.
Move to Method 3	For a text console over the USB-UART instead of a dashboard, see the companion <code>USB_UART.pdf</code> . The two are complementary: UART gives comfortable data entry, VIO+ILA gives cycle-level visibility.

18. Writing an XDC for any design — a general guide

Everything up to here has been specific to this accelerator. This section is not: it is a self-contained guide to writing Xilinx Design Constraints for **any** architecture on **any** 7-series or UltraScale part. The accelerator's own `vio_debug.xdc` is a deliberately tiny example — one pin, one clock — so it teaches almost nothing about the general problem. Most designs need far more.

THE MENTAL MODEL THAT MAKES XDC MAKE SENSE

An XDC file is

NOT

a configuration file. It is a

TCL SCRIPT

that Vivado executes top to bottom against an in-memory netlist. Three consequences follow, and nearly every beginner XDC bug traces to forgetting one of them:

1. ORDER MATTERS.

Later commands overwrite earlier ones. Two `create_clock`s on the same port means the second wins silently.

2. OBJECTS MUST EXIST WHEN THE LINE RUNS.

`get_ports` can only find a port that survived synthesis. If it was optimized away, `get_ports` returns empty and the `set_property` silently does nothing.

3. AN EMPTY `GET_*` IS NOT AN ERROR BY DEFAULT.

This is the single biggest trap — see §18.8.

18.1 The five jobs an XDC does

#	Job	Commands	Required?
1	Define clocks	<code>create_clock</code> , <code>create_generated_clock</code>	Always. Without this, nothing is timed and every path reports as unconstrained.
2	Place and configure I/O	<code>PACKAGE_PIN</code> , <code>IOSTANDARD</code> , <code>DRIVE</code> , <code>SLEW</code> , <code>PULLUP</code>	Always , for every top-level port that reaches a pin.
3	Constrain I/O timing	<code>set_input_delay</code> , <code>set_output_delay</code>	Whenever an interface must meet timing against something off-chip.
4	Declare timing exceptions	<code>set_false_path</code> , <code>set_multicycle_path</code> , <code>set_max_delay</code> , <code>set_clock_groups</code>	Whenever the default single-cycle assumption is wrong — CDC, resets, quasi-static config.
5	Set bitstream / config properties	<code>BITSTREAM.*</code> , <code>CONFIG_VOLTAGE</code> , <code>CFGBVS</code>	Often required for the bitstream to generate or the device to boot.

18.2 Clocks — get this right first

Every other timing constraint is meaningless without a clock definition. **Constrain the source, not the net.**

```
# --- a primary clock arriving on a package pin ---
create_clock -name sys_clk -period 10.000 [get_ports clk]           ;# 100 MHz, 50% duty

# --- explicit duty cycle / phase if not 50% ---
create_clock -name odd_clk -period 8.000 -waveform {0.000 3.000} [get_ports clk2]

# --- a differential pair: constrain the P side only ---
create_clock -name ref_clk -period 5.000 [get_ports clk_p]

# --- clocks OUT of an MMCM/PLL: do NOT hand-write these ---
#     Vivado derives them automatically from the primary clock.
#     Only use create_generated_clock for dividers you built in fabric:
create_generated_clock -name clk_div2 -source [get_pins reg/C] -divide_by 2 [get_pins div_reg/Q]
```

THREE CLOCK MISTAKES THAT COST DAYS

- **HAND-WRITING GENERATED CLOCKS FOR MMCM/PLL OUTPUTS.**

Vivado already creates them. Adding your own produces two overlapping definitions and nonsensical timing. Run `report_clocks` and look before you write.

- **CONSTRAINING A CLOCK NET INSTEAD OF THE SOURCE PORT/PIN.**

Use `[get_ports clk]` or `[get_pins .../0]`, not the buffer output net.

- **FORGETTING A CLOCK ENTIRELY.**

Unconstrained paths are simply *not analysed*. Timing then "passes" while being completely unverified — the most dangerous possible outcome. Always run `check_timing`.

18.3 I/O placement and standards

```
# preferred: one -dict per port, both properties together
set_property -dict {PACKAGE_PIN Y9  IOSTANDARD LVCMOS33} [get_ports clk]

# buses need per-bit constraints; note the braces around the index
set_property -dict {PACKAGE_PIN T22 IOSTANDARD LVCMOS33} [get_ports {led[0]}]
set_property -dict {PACKAGE_PIN T21 IOSTANDARD LVCMOS33} [get_ports {led[1]}]

# drive strength and slew (outputs only) - reduce both to cut ground bounce
set_property DRIVE 8      [get_ports {led[*]}]
set_property SLEW  SLOW  [get_ports {led[*]}]

# internal pull-up/down for a floating input
set_property PULLUP true [get_ports btn_n]
```

IOSTANDARD IS NOT COSMETIC — IT IS BANK-WIDE AND IT IS PHYSICAL

Every I/O bank has one `VCC0` supply, fixed by the board. Every pin in that bank must use an `IOSTANDARD` compatible with it. Assigning `LVCMOS33` to a pin in a 1.8 V bank is a DRC error at best and, if forced through, can damage hardware.

THIS IS WHY YOU TAKE PIN ASSIGNMENTS FROM THE BOARD VENDOR'S MASTER XDC RATHER THAN INVENTING THEM.

Note in this project's own files that the ZedBoard's LEDs are `LVCMOS33` while its switches and buttons are `LVCMOS25` — different banks, different supplies.

18.4 Where pin names actually come from

1. **The board vendor's master XDC.** Digilent, Avnet, Xilinx and others publish one per board, with every pin listed and commented out. **Download it, uncomment what you need, rename the ports to match your top module.** This is the correct workflow — never transcribe pin numbers from a PDF by hand.
2. **Board files.** If installed, selecting the board in the New Project wizard lets IP Integrator auto-assign board interfaces.
3. **The package file** (`<part>_pkg.txt`) for raw pin/bank/supply data.

CHECK THE BOARD REVISION

Pinouts change between board revisions. The ZedBoard is a good example — this project's own XDC carries an explicit warning to verify `PACKAGE_PIN` and `IOSTANDARD` against the master XDC for *your* revision. A wrong clock pin produces a bitstream that programs successfully and then does nothing at all, which is the hardest class of failure to diagnose.

18.5 I/O timing — the part most designs get wrong

Placing a pin says *where* a signal goes, not *when* it is valid. For any interface that must meet timing against an external device, you must model the outside world.

```
# SYSTEM-SYNCHRONOUS INPUT: external device and FPGA share a clock.
# Numbers come from the OTHER device's datasheet plus board trace delay.
set_input_delay -clock sys_clk -max 4.0 [get_ports {data_in[*]}]
set_input_delay -clock sys_clk -min 1.0 [get_ports {data_in[*]}]

# OUTPUT: -max is the downstream device's setup requirement + trace delay,
#         -min is negative of its hold requirement.
set_output_delay -clock sys_clk -max 3.0 [get_ports {data_out[*]}]
set_output_delay -clock sys_clk -min -1.0 [get_ports {data_out[*]}]

# SOURCE-SYNCHRONOUS: the clock travels WITH the data. Create a virtual
# clock for the external domain and reference that instead.
create_clock -name virt_src_clk -period 10.000
set_input_delay -clock virt_src_clk -max 2.0 [get_ports {src_data[*]}]

# Genuinely asynchronous or quasi-static pins (a DIP switch, a reset button)
# have no meaningful timing relationship - say so explicitly:
set_false_path -from [get_ports {sw[*]}]
set_false_path -from [get_ports rst_btn]
```

WHY THIS PROJECT'S XDC HAS NONE OF THESE

`vio_debug_top` has exactly one port — the clock. There is no data I/O to constrain, which is why `vio_debug.xdc` is three lines long. `fpga_top` (Method 1) does have switches, buttons and LEDs, but they are human-speed and asynchronous, so the correct constraint for them is `set_false_path`, not an input/output delay.

DO NOT TAKE THE BREVITY OF THESE FILES AS A MODEL FOR A REAL INTERFACE.

A DDR, Ethernet or ADC interface needs every one of the constraints above, taken from the peer device's datasheet.

18.6 Timing exceptions — and the discipline to use them sparingly

Command	Use when	Danger
<code>set_clock_groups -asynchronous</code>	Two clock domains have no phase relationship	The right tool for CDC. Cleaner and safer than scattering <code>set_false_path</code> . Does not remove the need for real synchronizers.
<code>set_false_path</code>	A path is logically impossible or genuinely doesn't matter	Turns timing analysis off for that path. Over-use is how designs "pass" timing and then fail in hardware.
<code>set_multicycle_path</code>	Logic is genuinely given N clocks to settle	Must constrain both setup and hold — <code>-setup N</code> then <code>-hold N-1</code> . Doing only setup is a classic silent hold violation.
<code>set_max_delay -datapath_only</code>	Async crossing that still needs a bounded skew	Preferred over a blanket false path for CDC data buses handed across by a handshake.

```
# the standard, safe CDC pattern
set_clock_groups -asynchronous \
  -group [get_clocks clk_100] \
  -group [get_clocks clk_50]

# a 2-cycle path, done properly - BOTH lines or you have a hold bug
set_multicycle_path 2 -setup -from [get_cells slow/*] -to [get_cells dest/*]
set_multicycle_path 1 -hold -from [get_cells slow/*] -to [get_cells dest/*]

# bounded async crossing (better than a false path for data)
set_max_delay 5.0 -datapath_only -from [get_cells src_reg] -to [get_cells dst_reg]
```

AN EXCEPTION IS A PROMISE YOU ARE MAKING TO THE TOOL

Every exception tells Vivado "do not check this". If the promise is wrong, the tool cannot catch the bug and you will find it as intermittent hardware misbehaviour that no simulation reproduces. Write the *reason* in a comment beside every exception. If you cannot state the reason, you do not yet know whether the exception is safe.

18.7 Configuration and bitstream properties

```
# Required on many 7-series boards or bitstream generation errors out:
set_property CFGBVS VCC0 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

# Allow unconstrained pins (PL-only Zynq, or during early bring-up).
# Use deliberately - it silences a DRC that is usually telling you something.
set_property BITSTREAM.GENERAL.UNCONSTRAINEDPINS {Allow} [current_design]

# Faster configuration from SPI flash
set_property BITSTREAM.CONFIG.SPI_BUSWIDTH 4 [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]

# Drive unused I/O to a defined state instead of leaving them floating
set_property BITSTREAM.CONFIG.UNUSEDPIN PULLDOWN [current_design]
```

18.8 The empty-object trap — the most important thing in this section

A CONSTRAINT ON AN OBJECT THAT DOES NOT EXIST DOES NOTHING, AND BY DEFAULT DOES NOT FAIL

```
set_property PACKAGE_PIN Y9 [get_ports clk] ;# typo: "clk"
```

`get_ports` returns an empty list, `set_property` applies to nothing, you get a warning buried in a log of hundreds, and the build completes. The pin is never assigned. Your design is silently wrong.

Defend against it in three ways:

```
# 1. Promote the warning to an error while developing
set_msg_config -id "Vivado 12-508" -new_severity ERROR ;# "No objects matched"

# 2. Assert explicitly for critical objects
if {[llength [get_ports -quiet clk]] == 0} { error "PORT clk NOT FOUND - check top module" }

# 3. Never use -quiet to hide a problem. It suppresses the warning too.
```

Then **always** run these three after implementation:

Command	What it catches
check_timing	Unconstrained clocks, missing I/O delays, unlocked registers — run this first, every time
report_clocks	Every clock Vivado <i>believes</i> exists, including ones it derived. Compare against what you intended.
report_cdc	Unsafe clock-domain crossings
report_timing_summary	WNS/WHs and, crucially, the count of unconstrained paths
report_exceptions	Exceptions that overlap, conflict, or match nothing

18.9 Organising constraints on a real project

One monolithic XDC becomes unmanageable. Split by concern, and control the order — remember the file is executed sequentially:

```
constraints/
├─ 01_clocks.xdc      create_clock, create_generated_clock, clock groups
├─ 02_pins.xdc        PACKAGE_PIN + IOSTANDARD (from the board master XDC)
├─ 03_io_timing.xdc   set_input_delay / set_output_delay
├─ 04_exceptions.xdc  false paths, multicycles, max_delay (MUST come after clocks)
└─ 05_bitstream.xdc   CFGBVS, CONFIG_VOLTAGE, BITSTREAM.*
```

Set `PROCESSING_ORDER` when a file must run at a specific stage:

```
set_property PROCESSING_ORDER EARLY [get_files 01_clocks.xdc]
set_property PROCESSING_ORDER LATE [get_files 04_exceptions.xdc]
```

Constraints can also be **scoped** to a module — essential for reusable IP, so an instance's constraints follow it wherever it is instantiated:

```
set_property SCOPED_TO_REF my_uart_core [get_files uart_timing.xdc]
set_property SCOPED_TO_CELLS u_uart_inst [get_files uart_timing.xdc]
```

18.10 A skeleton to start any new design from

```
##=====
## <design>.xdc -- <board>, part <partnumber>
## Pin assignments taken from <vendor> master XDC rev <X>. VERIFY.
##=====

## ---- 1. CLOCKS -----
create_clock -name sys_clk -period 10.000 [get_ports clk]
# derived MMCM/PLL clocks are created automatically - do not redefine

## ---- 2. CLOCK RELATIONSHIPS -----
# set_clock_groups -asynchronous -group [get_clocks sys_clk] \
#                               -group [get_clocks other_clk]

## ---- 3. PINS -----
set_property -dict {PACKAGE_PIN <PIN> IOSTANDARD <STD>} [get_ports clk]
# ... one line per port; buses need {name[i]} per bit

## ---- 4. I/O TIMING -----
# set_input_delay -clock sys_clk -max <t> [get_ports {din[*]}]
# set_output_delay -clock sys_clk -max <t> [get_ports {dout[*]}]
# async / human-speed pins:
# set_false_path -from [get_ports {sw[*]}]

## ---- 5. EXCEPTIONS (comment the REASON for every one) -----
# set_false_path -from [get_cells cfg_reg*] ;# quasi-static config

## ---- 6. BITSTREAM / CONFIG -----
set_property CFGBVS VCC0 [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

## ---- 7. DEBUG (only while debugging) -----
# set_property C_CLK_INPUT_FREQ_HZ 100000000 [get_debug_cores dbg_hub]
# set_property C_ENABLE_CLK_DIVIDER false [get_debug_cores dbg_hub]
```

18.11 Checklist before you trust a build

✓	Check
☐	Every clock is defined — <code>report_clocks</code> shows what you expect and nothing extra
☐	<code>check_timing</code> reports no unconstrained endpoints and no missing I/O delays
☐	Every top-level port has both <code>PACKAGE_PIN</code> and <code>IOSTANDARD</code>
☐	Every <code>IOSTANDARD</code> is compatible with its bank's <code>VCC0</code> on the real board
☐	Pin assignments came from the vendor master XDC for your board revision
☐	No <code>get_*</code> returned empty — search the log for "No objects matched"
☐	Every timing exception has a comment stating why it is safe
☐	Every <code>set_multicycle_path -setup</code> has a matching <code>-hold</code>
☐	Asynchronous domains are handled with <code>set_clock_groups</code> , and real synchronizers exist in the RTL
☐	WNS and WHS both positive , with zero unconstrained paths
☐	<code>report_cdc</code> shows no unsafe crossings

THE ONE HABIT WORTH FORMING

Run `check_timing` and read the "unconstrained" counts *before* you look at WNS. A design with WNS +2 ns and 4 000 unconstrained endpoints is in far worse shape than one with WNS +0.1 ns and zero. Slack on paths nobody analysed is not slack — it is ignorance with a positive number attached.

Appendix A — data packing reference

A.1 The three word formats

```
weight_mem[r]  bits  [31:24] [23:16] [15:8] [7:0]
                  W[r][3] W[r][2] W[r][1] W[r][0]      int8, lane c = COLUMN c

a_mem[i]        bits  [31:24] [23:16] [15:8] [7:0]
                  A[i][3] A[i][2] A[i][1] A[i][0]      int8, lane r = ROW r

output_mem[i]   bits  [127:96] [95:64] [63:32] [31:0]
                  C[i][3] C[i][2] C[i][1] C[i][0]      int32, lane c = COLUMN c
```

A.2 Packing an arbitrary matrix by hand

For a signed byte v , the hex nibble pair is v for $0...127$ and $v+256$ for $-128...-1$.

```
word = (byte3 << 24) | (byte2 << 16) | (byte1 << 8) | byte0
```

Worked example using the architecture document's reference matrices:

```
W = [  2  -1   3   0 ]   lane order in the word: [b3][b2][b1][b0]
    [  1   4  -2   5 ]   row 0 -> 00 03 FF 02 -> weight_mem[0] = 0x0003FF02
    [-3   2   1   1 ]   row 1 -> 05 FE 04 01 -> weight_mem[1] = 0x05FE0401
    [  0   1   2  -4 ]   row 2 -> 01 01 02 FD -> weight_mem[2] = 0x010102FD
                          row 3 -> FC 02 01 00 -> weight_mem[3] = 0xFC020100

A = [  1   2   3   4 ]   A0 -> 04 03 02 01 -> a_mem[0]      = 0x04030201
    [  5  -1   0   2 ]   A1 -> 02 00 FF 05 -> a_mem[1]      = 0x0200FF05
    [-2   3   1   1 ]   A2 -> 01 01 03 FE -> a_mem[2]      = 0x010103FE

expected C = [-5  17  10  -3 ] -> o_rdata = FFFF_FFFD 0000_000A 0000_0011 FFFF_FFFB
              [ 9  -7  21 -13 ] -> o_rdata = FFFF_FFF3 0000_0015 FFFF_FFF9 0000_0009
              [-4  17  -9  12 ] -> o_rdata = 0000_000C FFFF_FFF7 0000_0011 FFFF_FFFC
```

Note the sign extension: negative int32 results appear as `FFFF_FFxx`. The Tcl helper `accel_read` sign-extends for you.

A.3 T5 golden case quick card

Step	Probe	Value	Then
1	vio_w_wdata / vio_w_addr	04030201 / 0	toggle vio_w_we 0→1→0
2	vio_w_wdata / vio_w_addr	08070605 / 1	toggle vio_w_we 0→1→0
3	vio_w_wdata / vio_w_addr	0C0B0A09 / 2	toggle vio_w_we 0→1→0
4	vio_w_wdata / vio_w_addr	100F0E0D / 3	toggle vio_w_we 0→1→0
5	vio_a_wdata / vio_a_addr	00000001 / 0	toggle vio_a_we 0→1→0
6	vio_a_wdata / vio_a_addr	00000100 / 1	toggle vio_a_we 0→1→0
7	vio_a_wdata / vio_a_addr	00010000 / 2	toggle vio_a_we 0→1→0
8	vio_a_wdata / vio_a_addr	01010101 / 3	toggle vio_a_we 0→1→0
9	vio_num_vectors	04	act_base = out_base = 0
10	vio_start	0→1→0	done_sticky must go to 1
11	vio_o_addr = 0...3	read o_rdata	expect 1 2 3 4 / 5 6 7 8 / 9 10 11 12 / 28 32 36 40

Appendix B — full source listing of `vio_debug_top.sv`

```
`timescale 1ns/1ps
//=====
// vio_debug_top.sv -- VIO + ILA debug harness for the 4x4 systolic
//                      accelerator (Method 2, ZedBoard xc7z020clg484-1)
//
// The ONLY external pin is the 100 MHz PL oscillator (Y9). Every input
// to the accelerator comes from a Vivado VIO core over JTAG, and every
// output goes back to the same VIO dashboard. An ILA captures the run
// cycle-by-cycle. accelerator_system_top is instantiated UNMODIFIED.
//=====
module vio_debug_top (
    input logic clk                // 100 MHz PL oscillator, pin Y9
);
    //-----
    // Power-on reset: hold rst_n low for 65536 clocks (~655 us).
    //-----
    logic [15:0] por = '0;
    logic      rst_n;
    always_ff @(posedge clk)
        if (!(&por)) por <= por + 1'b1;
    assign rst_n = &por;

    //-----
    // VIO probe_out -> levels driven from the Hardware Manager dashboard
    //-----
    logic      vio_start_lvl;    // probe_out0
    logic [7:0] vio_num_vectors; // probe_out1
    logic [7:0] vio_act_base;    // probe_out2
    logic [7:0] vio_out_base;    // probe_out3
    logic [31:0] vio_w_wdata;    // probe_out4
    logic [1:0] vio_w_addr;      // probe_out5
    logic      vio_w_we_lvl;     // probe_out6
    logic [31:0] vio_a_wdata;    // probe_out7
    logic [7:0] vio_a_addr;      // probe_out8
    logic      vio_a_we_lvl;     // probe_out9
    logic [7:0] vio_o_addr;      // probe_out10

    //-----
    // Level -> single-cycle pulse
    //-----
    logic start_p, w_we_p, a_we_p;
    oneshot u_os_start (.clk(clk), .rst_n(rst_n), .lvl(vio_start_lvl), .pulse(start_p));
    oneshot u_os_wwe   (.clk(clk), .rst_n(rst_n), .lvl(vio_w_we_lvl), .pulse(w_we_p));
    oneshot u_os_awe   (.clk(clk), .rst_n(rst_n), .lvl(vio_a_we_lvl), .pulse(a_we_p));

    //-----
    // The accelerator (LEVEL 3) -- instantiated unmodified
    //-----
    logic [127:0] o_rdata;
    logic [2:0]   phase;
    logic         busy, done;
    logic signed [31:0] result [0:3];

    accelerator_system_top u_acc (
        .clk      (clk),
        .rst_n    (rst_n),
        .start     (start_p),
        .num_vectors (vio_num_vectors),
        .act_base  (vio_act_base),
        .out_base  (vio_out_base),
        .host_a_we  (a_we_p),
        .host_a_addr (vio_a_addr),
        .host_a_wdata (vio_a_wdata),
        .host_w_we  (w_we_p),
        .host_w_addr (vio_w_addr),
    );
endmodule
```

```

        .host_w_data (vio_w_data),
        .host_o_addr (vio_o_addr),
        .o_rdata     (o_rdata),
        .phase       (phase),
        .busy        (busy),
        .done        (done),
        .result       (result)
    );

//-----
// Sticky done + completed-job counter
//-----
    logic         done_sticky;
    logic [15:0] done_count;
    always_ff @(posedge clk) begin
        if (!rst_n) begin
            done_sticky <= 1'b0;
            done_count  <= '0;
        end else begin
            if (start_p) done_sticky <= 1'b0;    // arm for the new job
            else if (done) done_sticky <= 1'b1;  // latch the 10 ns pulse
            if (done) done_count <= done_count + 1'b1;
        end
    end

// `result` is an unpacked array; flatten it so VIO/ILA can probe it.
    logic [127:0] result_flat;
    assign result_flat = {result[3], result[2], result[1], result[0]};

//-----
// VIO (generate as "vio_0")
//-----
    vio_0 u_vio (
        .clk          (clk),
        .probe_in0    (busy),           // 1
        .probe_in1    (done_sticky),    // 1
        .probe_in2    (phase),          // 3
        .probe_in3    (o_rdata),        // 128
        .probe_in4    (result_flat),    // 128
        .probe_in5    (done_count),     // 16
        .probe_out0    (vio_start_lvl),  // 1
        .probe_out1    (vio_num_vectors), // 8   initial value 0x04
        .probe_out2    (vio_act_base),   // 8
        .probe_out3    (vio_out_base),   // 8
        .probe_out4    (vio_w_data),     // 32
        .probe_out5    (vio_w_addr),     // 2
        .probe_out6    (vio_w_we_lvl),   // 1
        .probe_out7    (vio_a_data),     // 32
        .probe_out8    (vio_a_addr),     // 8
        .probe_out9    (vio_a_we_lvl),   // 1
        .probe_out10   (vio_o_addr)     // 8
    );

//-----
// ILA (generate as "ila_0"), trigger on probe0 == 1
//-----
    ila_0 u_ila (
        .clk          (clk),
        .probe0       (start_p),        // 1   <- trigger on this
        .probe1       (busy),           // 1
        .probe2       (done),           // 1
        .probe3       (phase),          // 3
        .probe4       (w_we_p),         // 1
        .probe5       (a_we_p),         // 1
        .probe6       (vio_a_addr),     // 8
        .probe7       (o_rdata),        // 128
        .probe8       (result_flat),    // 128
        .probe9       (done_count)     // 16
    );
endmodule

```

```

//=====
// oneshot -- convert a VIO probe_out level into a single-clock pulse on
// its rising edge. Without this, a held-high `start` relaunched the
// 17-cycle job ~4 million times per second and the ILA never settles.
//=====
module oneshot (
    input  logic clk,
    input  logic rst_n,
    input  logic lvl,
    output logic pulse
);
    logic lvl_q;
    always_ff @(posedge clk) begin
        if (!rst_n) begin
            lvl_q <= 1'b0;
            pulse <= 1'b0;
        end else begin
            lvl_q <= lvl;
            pulse <= lvl & ~lvl_q;
        end
    end
end
endmodule

```

Design reference: [context_mkdown/](#) — chunk 07 (controller FSM), chunk 11 (L3 wrapper and host protocol), chunk 12 (dataflow and timing derivations), chunk 13 (worked example / T5 golden case), chunk 06 (memory word and lane layout).

Functional baseline: 11/11 testbenches, 3 680 checks, 0 mismatches ([verification_recheck/](#)).

Companion document: [USB_UART.pdf](#) — Method 3, the text console over the on-board USB-UART.